OnBuch+

SIGNED COURSE • PREMIUM

COMPUTER SCIENCE • UPPER SIXTH | UPPER SIXTH — COMPUTER SCIENCE

CHAPTER 19

NETWORKING, SECURITY, SOFTWARE ENGINEERING & SYNTHESIS

SUGGESTED TIME: 3 PERIODS

IN THIS CHAPTER

This final chapter of Upper Sixth Computer Science brings together data communication and networks, security threats and countermeasures, the software engineering life cycle, and a hands-on subnetting and LAN-simulation lab. It consolidates the whole A-Level programme and prepares students for the synthesis questions of the GCE Advanced Level paper.

- By the end of this chapter I can explain how data is transmitted (signals, media, multiplexing, switching) and describe the OSI and TCP/IP models layer by layer.
- By the end of this chapter I can classify networks (LAN, WAN, MAN, PAN), identify topologies and network devices, and justify design choices for a given organisation.
- By the end of this chapter I can perform IPv4 addressing and subnetting calculations (network address, broadcast, host range, CIDR) for a real institution.
- By the end of this chapter I can identify major security threats (malware, phishing, interception, DoS) and match them with appropriate countermeasures including encryption and authentication.
- By the end of this chapter I can describe the phases of the SDLC and compare life-cycle models (waterfall, iterative, agile, prototyping).
- By the end of this chapter I can explain software testing levels (unit, integration, system, acceptance) and the role of documentation and maintenance.

DOWNLOAD ONBUCH
Free • Google Play
AI tutor, past papers, results

GO ONBUCH+
All signed courses, unlimited AI tutor, PDF sheets — OnBuch+ tab in the app

THIS SIGNED COURSE CONTAINS

COURSE complete, illustrated, step by step

METHODS and worked examples

EXERCISES exam style + solutions

SUMMARY SHEET the essentials to remember

Signed • OnBuch+
ONBUCH+ TEACHING TEAM
FOLLOWS THE OFFICIAL MINESEC SYLLABUS

OnBuch+

CONTENTS

1. Data Communication & Network Fundamentals
2. IP Addressing & Network Security
3. Software Engineering, SDLC & Theoretical Computer Science
4. Hands-on Networking Lab: Subnetting & LAN Simulation
5. Integration activity
6. Methods and worked examples
7. Exercises
8. Solutions to the exercises
9. Summary sheet

CHAPTER OBJECTIVES

- By the end of this chapter I can explain how data is transmitted (signals, media, multiplexing, switching) and describe the OSI and TCP/IP models layer by layer.
- By the end of this chapter I can classify networks (LAN, WAN, MAN, PAN), identify topologies and network devices, and justify design choices for a given organisation.
- By the end of this chapter I can perform IPv4 addressing and subnetting calculations (network address, broadcast, host range, CIDR) for a real institution.
- By the end of this chapter I can identify major security threats (malware, phishing, interception, DoS) and match them with appropriate countermeasures including encryption and authentication.
- By the end of this chapter I can describe the phases of the SDLC and compare lifecycle models (waterfall, iterative, agile, prototyping).
- By the end of this chapter I can explain software testing levels (unit, integration, system, acceptance) and the role of documentation and maintenance.
- By the end of this chapter I can situate theoretical computer science notions (algorithms, complexity, computability, automata) within the A-Level programme.
- By the end of this chapter I can configure and simulate a small LAN with subnets in a tool such as Cisco Packet Tracer and verify connectivity with ping and ipconfig/ifconfig.

PREREQUISITES

- Binary and hexadecimal number systems, conversions and binary arithmetic (Lower Sixth).
- Basic hardware components of a computer system and the role of the operating system.
- Fundamentals of algorithms and programming (variables, control structures, testing a simple program).
- Elementary notions of the Internet, URLs, e-mail and web browsing from everyday use and earlier classes.

1 DATA COMMUNICATION & NETWORK FUNDAMENTALS

At a busy cybercafé in Buea, a student sends her GCE registration details over the shop's Wi-Fi while, unknown to her, another customer runs a packet sniffer on the same network. Meanwhile, the cybercafé owner wonders why his /24 network cannot serve his three branches in Molyko, Great Soppo and Bonduma without buying new equipment, and a local start-up's school-fees app keeps crashing because it was coded without any testing plan. Networks, security and disciplined software development are not abstract topics: they decide whether Cameroon's digital services can be trusted. This chapter gives you the tools to understand, secure and build them.

1.1 The Data Communication Model

Data communication is the exchange of data (in the form of 0s and 1s) between two devices via a transmission medium. For communication to occur, five components must work together.

DEFINITION **Data Communication**

Data communication is the process of transferring digital or analogue data between a source and a destination through a transmission medium. It involves a source (sender), a transmitter, a transmission medium, a receiver and a destination.

The five components are:

1. **Source (Sender):** The device that generates the message (e.g., a computer, smartphone, sensor).
2. **Transmitter:** Converts the message into signals suitable for the medium (e.g., a modem modulates digital data onto analogue carrier waves; a NIC encodes bits as voltage pulses or light).
3. **Transmission Medium:** The physical path carrying the signal (copper cable, fibre optic, air for wireless).
4. **Receiver:** Converts the received signals back into a message (demodulation, decoding).
5. **Destination:** The device that consumes the message (e.g., a server, printer, another computer).

For a network to be effective, it must meet four fundamental criteria:

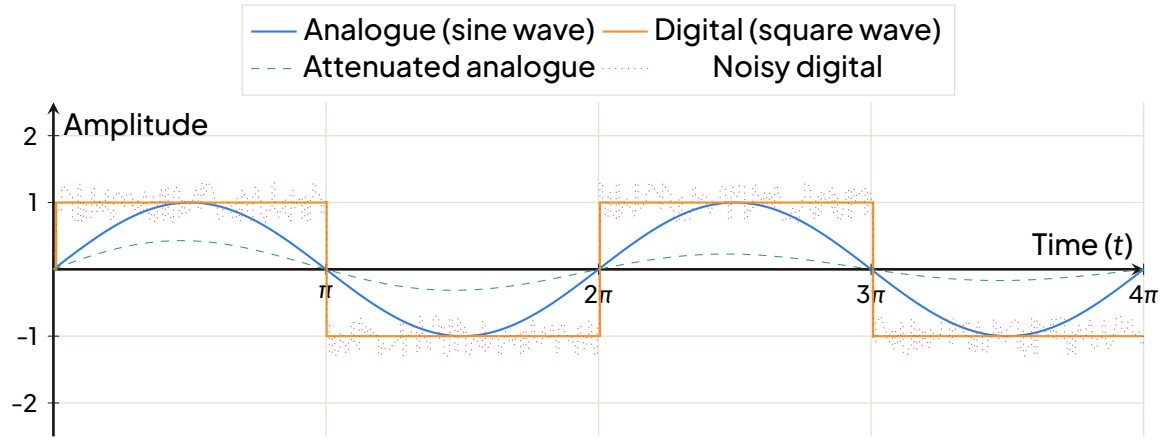
- **Delivery:** Data must reach the correct destination; only the intended recipient receives it.
- **Accuracy:** Data must arrive unchanged; corruption (bit errors) must be detected and corrected.
- **Timeliness:** Data must be delivered within an acceptable delay. For real-time traffic (VoIP, video conferencing), late data is useless.
- **Jitter:** The variation in packet arrival time. Low jitter is critical for streaming and voice; high

jitter causes choppy audio/video.

1.2 Signals, Transmission Modes and Media

1.3 Analogue vs Digital Signals

Information is carried by signals. An **analogue signal** is continuous in both time and amplitude (e.g., human voice, radio waves). A **digital signal** is discrete, taking only a finite set of values (typically two voltage levels representing 0 and 1).



Analogue (smooth sine) vs Digital (square) signals. Dashed green shows attenuation (amplitude loss over distance). Dotted pink illustrates noise distorting the digital signal.

Key signal impairments:

- **Attenuation:** Loss of signal strength over distance. Amplifiers (analogue) or repeaters (digital) are needed.
- **Noise:** Unwanted signals added to the original. Types: thermal noise, crosstalk, impulse noise. Signal-to-Noise Ratio (SNR) measures quality.
- **Distortion:** Change in signal shape due to different frequency components travelling at different speeds (dispersion).

Bandwidth (in Hz) is the range of frequencies a medium can carry. **Data rate** (in bits per second, bps) is the speed of data transfer. By Nyquist and Shannon, maximum data rate depends on bandwidth and SNR.

1.4 Transmission Modes

The direction of signal flow defines the transmission mode:

- **Simplex:** One direction only (e.g., radio broadcast, keyboard → CPU).
- **Half-Duplex:** Both directions, but not simultaneously (e.g., walkie-talkie, legacy Ethernet hub).
- **Full-Duplex:** Both directions simultaneously (e.g., telephone, modern switched Ethernet,

fibre optic). Requires two physical channels or frequency division.

1.5 Transmission Media

Media are classified as **guided** (wired) or **unguided** (wireless).

Medium	Description	Typical Use / Cameroon Context	Max Data Rate (Typical)
Twisted Pair (UTP/STP)	Copper wires twisted to reduce crosstalk. Cat 5e/6/6a/7.	LANs in schools, offices, homes. CAMTEL last-mile copper.	1 Gbps – 10 Gbps (short range)
Coaxial Cable	Central conductor, insulation, braided shield.	Cable TV (Canal+, CANAL+), legacy LANs, CCTV.	100 Mbps – 1 Gbps (DOCSIS 3.1 higher)
Fibre Optic	Light pulses in glass/plastic core. Single-mode (long haul) vs Multi-mode (campus).	**CAMTEL fibre backbone** (Yaoundé–Douala–Bafoussam), ISP backbones, FTTH.	10 Gbps – 100+ Gbps (Tbps in labs)
Radio Waves	Omnidirectional, low frequency.	FM radio, Bluetooth, Wi-Fi (2.4/5/6 GHz), LoRaWAN for IoT farms.	Up to 9.6 Gbps (Wi-Fi 6/6E)
Microwave	Line-of-sight, high frequency. Towers ≈ 30–50 km apart.	Backhaul links between mountain tops (e.g., Mount Cameroon relays), campus links.	100 Mbps – 10 Gbps
Satellite	Geostationary (GEO, high latency ≈ 600 ms) or LEO (Starlink, low latency).	**VSAT in rural areas** (East, Adamawa, Far North) for banking, admin, schools; Starlink emerging.	10–500 Mbps (LEO)

DID YOU KNOW? **Cameroon's Digital Backbone**

CAMTEL manages the national fibre optic backbone (Backbone National à Fibre Optique) connecting major cities. The ACE (Africa Coast to Europe) and WACS (West Africa Cable System) submarine cables land at Kribi and Limbe, giving Cameroon international bandwidth. Rural connectivity often relies on VSAT (Very Small Aperture Terminal) or microwave links where fibre is not yet deployed.

1.6 Network Classification, Architectures and Topologies

1.7 Classification by Scale

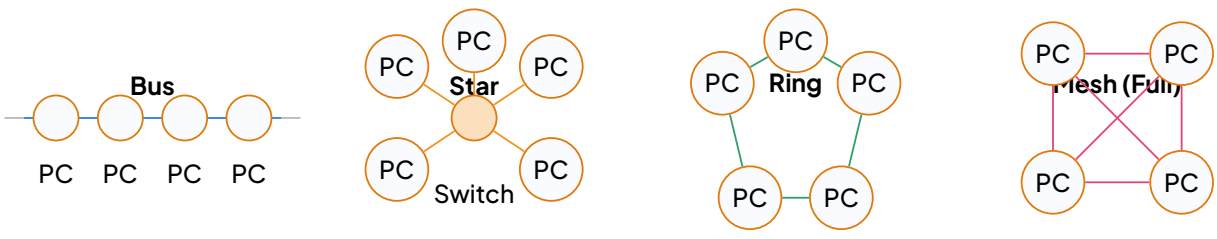
- **PAN (Personal Area Network):** ≤ 10 m. Bluetooth, USB, Zigbee (e.g., phone ↔ headset).
- **LAN (Local Area Network):** Single building or campus (≤ 2 km). High speed (Gbps), low latency. Ethernet, Wi-Fi.
- **MAN (Metropolitan Area Network):** City-wide (e.g., Yaoundé metropolitan ring). Often fibre or microwave interconnecting LANs.
- **WAN (Wide Area Network):** Large geographical area (country, continent, world). The Internet is the ultimate WAN. Leased lines, MPLS, VPNs, satellite.

1.8 Network Architectures

- **Client-Server:** Centralised servers provide resources/services (files, database, printing, web) to clients. Scalable, manageable, secure, but single point of failure (mitigated by clustering). Example: School management system (server) accessed by admin clients.
- **Peer-to-Peer (P2P):** All nodes act as both clients and servers. Decentralised, easy to set up, but hard to secure/manage at scale. Example: File sharing between laptops in a classroom ad-hoc network.

1.9 Network Topologies

Topology defines the physical or logical layout of connections.



Common network topologies: Bus, Star, Ring, and Full Mesh.
Nodes represent devices; central node in Star is a Switch.

Topology	Advantages	Disadvantages	Fault Tolerance
Bus	Simple, low cable cost, easy to extend.	Collisions (CSMA/CD), whole network down if backbone breaks, hard to troubleshoot.	Low (backbone cut = total failure)
Star	Easy to manage, isolate faults, add/remove nodes, high performance (switch).	Central switch failure kills network, more cable than bus.	High (single link cut ≠ network down)
Ring	Ordered access (token), no collisions, predictable performance.	One break stops ring (unless dual ring), adding/removing disrupts network.	Low (single break) / High (dual ring/FDDI)
Mesh (Full)	Maximum redundancy, privacy, traffic distribution.	Very expensive ($n(n-1)/2$ links), complex installation.	Very High (multiple paths)
Tree (Hierarchical)	Scalable, combines star/bus, good for WANs.	Root failure partitions network, complex cabling.	Medium (depends on root/branch redundancy)

METHOD **Choosing a Topology and Media for a Scenario**

1. Analyse requirements: Number of nodes, geographic spread, budget, required bandwidth, criticality (fault tolerance).
2. Small office / classroom (< 30 nodes, one room): Star topology with Cat 6 UTP (1 Gbps) or Cat 6a (10 Gbps). Cheap, easy to manage.
3. School campus (multiple buildings): Tree (hierarchical) topology. Backbone: Single-mode fibre between buildings (core switches). Building floors: Star with Cat 6a / Multi-mode fibre to access switches.
4. Critical server farm / Data centre: Partial Mesh or Leaf-Spine (modern Clos topology) with Fibre (OM4/OM5) for high throughput and redundancy.
5. Rural branch office (no fibre): Point-to-point Microwave or VSAT link to HQ (WAN). Local LAN remains Star.
6. Ad-hoc meeting / IoT sensors: Mesh (wireless mesh like Zigbee/Thread) or Star (Wi-Fi AP).

1.10 Network Devices and their Layers

Devices operate at specific layers of the OSI model. Understanding this is crucial for troubleshoot-

ing and design.

Device	OSI Layer	Role / Key Characteristic
NIC (Network Interface Card)	1 (Physical) / 2 (Data Link)	Physical connection (MAC address), framing, media access.
Repeater / Hub	1 (Physical)	Regenerates/amplifies signals. **Hub = multi-port repeater**. Broadcasts to **ALL** ports (collision domain = all ports). Obsolete for LANs.
Bridge / Switch	2 (Data Link)	Learns MAC addresses (MAC table). **Forwards frames only to destination port** (micro-segmentation). Each port = separate collision domain. Switch = multi-port bridge.
Router	3 (Network)	Routes **packets** based on **IP addresses**. Connects different networks (LAN to WAN, VLANs). Blocks broadcasts.
Modem (Modulator-Demodulator)	1 (Physical)	Converts digital ↔ analogue (DSL, Cable, Dial-up, Fibre ONT).
Access Point (AP)	1 / 2	Wireless bridge: connects wireless clients (802.11) to wired LAN (802.3).
Firewall	3 / 4 / 7	Filters traffic by IP, port, application (stateful inspection, DPI, IPS). Can be hardware appliance or software.
Gateway	4–7	Protocol converter (e.g., Email gateway SMTP ↔ X.400, VoIP gateway).

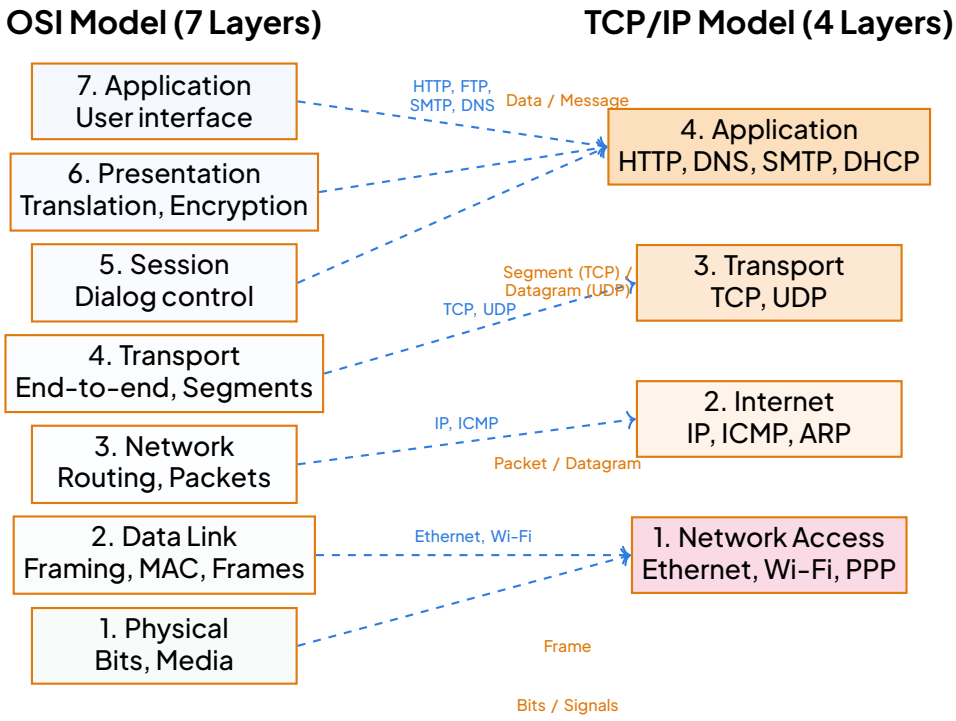
WARNING **Hub vs Switch – Classic Exam Trap**

Hub (Layer 1): "Dumb" device. Receives bits on one port, regenerates and sends out ALL other ports. Creates ONE large collision domain. Bandwidth is shared. Half-duplex only.

Switch (Layer 2): "Intelligent" device. Learns MAC addresses. Forwards frame ONLY to the port where destination MAC resides (unicast). Creates SEPARATE collision domain per port. Supports full-duplex. Never say "switch broadcasts" (it floods unknown unicast, but does not broadcast known unicast).

1.11 The OSI and TCP/IP Reference Models

Models standardise communication functions into layers. The **OSI Model** (7 layers) is the theoretical reference; the **TCP/IP Model** (4 layers) is the practical implementation suite of the Internet.



Mapping between OSI and TCP/IP models with example protocols and Protocol Data Units (PDUs) at each layer.

1.12 OSI Layer Functions (Bottom-Up)

1. **Physical (Layer 1):** Transmission of raw bits over a medium. Defines voltages, pinouts, cables, modulation. **PDU: Bit.** Devices: Hub, Repeater, Modem, NIC (phy).
2. **Data Link (Layer 2):** Node-to-node delivery. Framing, physical addressing (MAC), error detection (CRC), flow control, media access (CSMA/CD, CSMA/CA). **PDU: Frame.** Devices: Switch, Bridge, NIC (MAC), AP.
3. **Network (Layer 3):** Source-to-destination delivery across multiple networks. Logical ad-

dressing (IP), routing, fragmentation, congestion control. **PDU: Packet.** Device: Router. Protocols: IP, ICMP, ARP, OSPF, BGP.

4. **Transport (Layer 4):** Process-to-process delivery (Port numbers). Segmentation/reassembly, connection control (TCP), reliability, flow control. **PDU: Segment (TCP), Datagram (UDP).** Protocols: TCP, UDP, SCTP.
5. **Session (Layer 5):** Dialog control, synchronisation, checkpointing. (Often merged into Application in TCP/IP). Protocols: NetBIOS, RPC, PPTP.
6. **Presentation (Layer 6):** Translation, encryption/decryption, compression. (Merged into Application). Protocols: SSL/TLS (partly), JPEG, MPEG, ASCII/EBCDIC.
7. **Application (Layer 7):** Network services to user applications. **PDU: Data/Message.** Protocols: HTTP, HTTPS, FTP, SMTP, POP3, IMAP, DNS, DHCP, SSH, SNMP.

1.13 Encapsulation and PDUs

As data moves down the stack, each layer adds a header (and sometimes a trailer) to the PDU from the layer above.

$$\text{Data} \xrightarrow{\text{L7–L5}} \text{Data} \xrightarrow{\text{L4 (TCP)}} \text{TCP Header} + \text{Data} = \text{Segment} \xrightarrow{\text{L3 (IP)}} \text{IP Header} + \text{Segment} = \text{Packet} \xrightarrow{\text{L2 (Ethernet)}} \text{E}$$

At the destination, the process is reversed (**decapsulation**).

KEY POINTS **Exam Focus: One Protocol Per Layer**

GCE examiners frequently ask: "Name a protocol operating at Layer X of the OSI/TCP/IP model." Memorise at least one solid example per layer:

- L7: HTTP (or HTTPS, FTP, SMTP, DNS, DHCP)
- L4: TCP (reliable) or UDP (unreliable)
- L3: IP (IPv4/IPv6), ICMP, ARP
- L2: Ethernet (802.3), Wi-Fi (802.11), PPP, HDLC
- L1: RS-232, V.35, 802.3 PHY, DSL, GPON

For TCP/IP model: Application (HTTP), Transport (TCP/UDP), Internet (IP), Network Access (Ethernet).

DEFINITION **Protocol**

A protocol is a set of rules and conventions governing the exchange of data between entities. It defines syntax (format, signal levels), semantics (meaning, error handling, control), and timing (synchronisation, speed matching). Standards (e.g., IEEE 802.3, IETF RFCs) ensure interoperability between vendors.

1.14 Key Protocols and Switching Techniques

1.15 Application Layer Protocols (TCP/IP)

Protocol	Port(s)	Transport	Purpose
HTTP / HTTPS	80 / 443	TCP	Web browsing. HTTPS = HTTP + TLS/SSL (encryption, authentication).
FTP	20 (data), 21 (ctrl)	TCP	File transfer. Uses two connections. Insecure (clear text). Use SFTP/FTPS.
SMTP	25 (587 sub)	TCP	Sending email (Mail Transfer Agent → MTA).
POP3	110 (995 SSL)	TCP	Retrieving email (download & delete from server). Simple, offline access.
IMAP	143 (993 SSL)	TCP	Retrieving email (sync folders, server-side storage). Multiple clients.
DNS	53	UDP (mostly) / TCP	Domain name ↔ IP resolution. Hierarchical, distributed database.
DHCP	67 (srv), 68 (cli)	UDP	Dynamic IP configuration (IP, mask, gateway, DNS). DORA process.

1.16 Transport Layer: TCP vs UDP

Feature	TCP (Transmission Control Protocol)	UDP (User Datagram Protocol)
Connection	Connection-oriented (3-way handshake)	Connectionless (fire & forget)
Reliability	Reliable (ACKs, retransmission, sequencing)	Unreliable (no guarantee, no order)
Flow Control	Yes (Sliding Window)	No
Congestion Control	Yes (Slow Start, AIMD)	No (application must handle)
Header Size	20–60 bytes	8 bytes (low overhead)
Use Cases	Web (HTTP), Email (SMTP/IMAP), File (FTP), SSH, Database	Streaming (RTP), VoIP, DNS, Gaming, DHCP, SNMP, TFTP

1.17 Circuit Switching vs Packet Switching

This is a fundamental architectural distinction.

PROPERTY **Circuit Switching vs Packet Switching**

Criterion	Circuit Switching	Packet Switching
Concept	Dedicated physical path (circuit) established before communication (setup phase). Path held for duration.	No dedicated path. Data split into packets (header + payload). Each packet routed independently.
Resource Reservation	Reserved end-to-end (guaranteed bandwidth).	On-demand (statistical multiplexing). Shared links.
Delay	Constant (propagation only after setup). Setup delay high.	Variable (queuing delay at routers). No setup delay.
Efficiency	Low (silence → wasted bandwidth).	High (bursty traffic shares link).
Reliability	High (dedicated path). Failure breaks circuit.	Robust (packets rerouted around failure).
Store-and-Forward	No (real-time stream).	Yes (routers buffer packets).
Examples	PSTN (traditional telephone), ISDN, SONET/SDH, GSM voice (circuit-switched core).	Internet (IP), Ethernet, Frame Relay, ATM (cells), X.25.

Note: Modern networks (4G/5G LTE, VoIP) use packet switching for everything (voice encapsulated in IP packets). Circuit switching is largely legacy for core voice, though optical circuit switching (OTN) exists for high-capacity backbone pipes.

WORKED EXAMPLE 1 **Analysing a Communication Scenario**

Statement: A teacher in Yaoundé uses a video conferencing app (UDP/RTP based) to teach a class in Maroua. The link uses the CAMTEL fibre backbone. During the lesson, a fibre cut occurs between Ngaoundéré and Garoua. The routing protocol (OSPF) converges in 2 seconds, rerouting traffic via an alternate microwave link. The video freezes for 3 seconds then continues with slightly lower quality.
Questions:

1. Identify the switching technique used. Justify.
2. At which OSI layer does the routing protocol (OSPF) operate? What is its PDU?
3. The video uses UDP. Why is UDP preferred over TCP for live video?

4. If the teacher downloads the recorded lesson later via HTTPS, identify the Application, Transport, Network and Data Link protocols involved.

SOLUTION

Detailed Solution:

1. Packet Switching. Justification: The traffic is rerouted dynamically around a failure (fibre cut) without a dedicated end-to-end circuit. Packets are independently routed (OSPF convergence). Statistical multiplexing allows sharing the microwave link.
2. Layer 3 (Network Layer). OSPF (Open Shortest Path First) is a routing protocol. It runs directly over IP (protocol number 89). Its PDU is an OSPF Packet encapsulated in an IP Packet.
3. UDP is preferred because: (a) No connection setup delay (low latency start). (b) No head-of-line blocking: a lost packet is skipped; TCP would retransmit, delaying subsequent frames and increasing jitter. (c) No congestion control throttling the stream; the application (RTP) handles rate adaptation. (d) Lower header overhead (8 bytes vs 20+ bytes).
4. Application: HTTPS (HTTP over TLS). Transport: TCP (port 443). Network: IPv4 or IPv6. Data Link: Ethernet (802.3) on wired LAN / Wi-Fi (802.11) on wireless / GPON on fibre access. (Physical: Light pulses on fibre).

EXERCISE 1 **Practice Questions**

1. A cybercafé in Douala uses a single 24-port hub to connect 20 PCs. Users complain of slow file transfers. Explain the cause using the concepts of collision domain and duplex mode. What device should replace the hub?
2. List the 7 layers of the OSI model from top to bottom. For each layer, give the name of the PDU and one device (or protocol) that operates primarily at that layer.
3. Compare the bandwidth, latency, and typical use case of: (a) Single-mode fibre, (b) Cat 6 UTP, (c) VSAT (GEO satellite), (d) Microwave link.
4. A student sends an email with an attachment from her phone (Wi-Fi) to a lecturer's university mail server. Trace the encapsulation process from the Application layer down to the Physical layer at the sender, and decapsulation at the receiver. Mention headers added at each stage.
5. Why does a router not forward broadcast frames (e.g., ARP requests) between interfaces, whereas a switch floods them? Relate to OSI layers and addressing.

SOLUTION **Exercise 1**

The hub operates at Layer 1. It creates one single collision domain for all 20 PCs. Only one device can transmit at a time (half-duplex). Collisions are frequent (CSMA/CD), causing retransmissions and low throughput. Solution: Replace the hub with a Layer 2 Switch. A switch creates a separate collision domain per port, enables full-duplex, and forwards frames based on MAC addresses, eliminating collisions between different conversations.

SOLUTION **Exercise 2**

1. Application (L7): PDU = Data/Message. Protocol: HTTP.
2. Presentation (L6): PDU = Data. Protocol: TLS/SSL, JPEG.
3. Session (L5): PDU = Data. Protocol: NetBIOS, RPC.
4. Transport (L4): PDU = Segment (TCP) / Datagram (UDP). Protocol: TCP, UDP.
5. Network (L3): PDU = Packet/Datagram. Device: Router. Protocol: IP, OSPF.
6. Data Link (L2): PDU = Frame. Device: Switch, NIC. Protocol: Ethernet, Wi-Fi.
7. Physical (L1): PDU = Bits/Signals. Device: Hub, Repeater, Cable. Standard: 802.3 PHY.

SOLUTION **Exercise 3**

- (a) Single-mode Fibre: Bandwidth: Very High (100+ Gbps/km). Latency: Very Low (≈ 5 μs/km). Use: Backbone, long-haul, FTTH, DC interconnect.
- (b) Cat 6 UTP: Bandwidth: 1 Gbps (100m), 10 Gbps (55m). Latency: Low. Use: LAN horizontal cabling, patch cords.
- (c) VSAT GEO: Bandwidth: Moderate (10–100 Mbps shared). Latency: Very High (≈ 600 ms RTT). Use: Rural/remote connectivity, maritime, backup.
- (d) Microwave: Bandwidth: High (100 Mbps – 10+ Gbps). Latency: Low (speed of light, LOS). Use: Backhaul, campus links, last-mile where fibre impossible.

SOLUTION **Exercise 4**

Sender (Encapsulation):

1. L7 (App): User data (SMTP commands + MIME attachment). Header: SMTP.
2. L6/L5: TLS encryption (if STARTTLS/SMTPS). Header: TLS Record.
3. L4 (Transport): TCP Segment. Header: Src Port (ephemeral), Dst Port (25/465/587), Seq, Ack, Flags, Window, Checksum.
4. L3 (Network): IP Packet. Header: Src IP (phone), Dst IP (mail server), TTL, Protocol=6 (TCP), Checksum.
5. L2 (Data Link): Wi-Fi Frame (802.11). Header: Frame Control, Duration, Addr1 (AP MAC), Addr2 (Phone MAC), Addr3 (Router MAC), Seq Ctrl. Trailer: FCS (CRC).
6. L1 (Physical): Bits modulated as OFDM symbols on 2.4/5/6 GHz carrier (802.11n/ac/ax).

Receiver (Decapsulation): Reverse process at each hop (AP → Router → … → Mail Server). At each router: L2 frame stripped, L3 IP header inspected (TTL−, routing table lookup), new L2 frame built for next hop. At Mail Server: L2 stripped, L3 stripped, L4 TCP reassembled, TLS decrypted, SMTP parsed, attachment saved.

SOLUTION **Exercise 5**

A router operates at Layer 3 (Network). It makes forwarding decisions based on IP addresses (logical addressing). Broadcast frames (e.g., ARP Request: FF:FF:FF:FF:FF:FF) are Layer 2 constructs. A router terminates Layer 2 domains: it strips the incoming frame, processes the packet, and builds a new frame for the outgoing interface. It does not forward the original Layer 2 broadcast frame. A switch operates at Layer 2 (Data Link). It forwards frames based on MAC addresses. A broadcast frame (Dest MAC = FF:FF:FF:FF:FF:FF) is, by definition, destined for all ports in the broadcast domain (the VLAN). The switch floods it out all ports (except ingress) to deliver it to all hosts in that Layer 2 segment.

2 IP ADDRESSING & NETWORK SECURITY

In the previous section we saw how data physically travels across networks: signals, media, topologies and protocols such as TCP/IP. But for a packet leaving a computer in Yaoundé to reach a server in Douala — or anywhere in the world — every machine must have a unique postal address on the network, and the journey must be protected against eavesdroppers and attackers. That is exactly the double subject of this section: **IP addressing**, which identifies machines, and **network security**, which protects them.

2.1 IPv4 Address Structure

DEFINITION **IP address**

An **IP address** (Internet Protocol address) is a unique numerical identifier assigned to every device (host) connected to a network that uses the Internet Protocol. In version 4 (IPv4), it is a 32-bit binary number, usually written in **dotted-decimal notation**: four blocks of 8 bits (octets), each converted to a decimal value between 0 and 255, separated by dots. Example: `192.168.1.10`.

Why dotted decimal? Because humans find `11000000.10101000.00000001.00001010` unreadable, while `192.168.1.10` is easy to write and remember. Each octet is simply a byte: the largest value $11111111_2 = 255$.

An IP address has two logical parts:

- the **network part**, which identifies the network itself (like the name of a quarter in Yaoundé);
- the **host part**, which identifies a particular machine inside that network (like the number of a house in the quarter).

DEFINITION **Subnet mask**

A **subnet mask** is a 32-bit number, also written in dotted decimal, that separates the network part from the host part of an IP address: bits set to `1` mark the network portion, bits set to `0` mark the host portion. Example: mask `255.255.255.0` means the first 24 bits identify the network and the last 8 bits identify the host.

Address classes.

Historically, IPv4 addresses were divided into five classes, determined by the first bits of the first octet:

Class	First octet	Default mask	Use
A	1 – 126	255.0.0.0 (/8)	Very large networks: 2^{24} - 2 hosts
B	128 – 191	255.255.0.0 (/16)	Medium networks: 2^{16} - 2 hosts
C	192 – 223	255.255.255.0 (/24)	Small networks: 2^8 - 2 = 254 hosts
D	224 – 239	—	Multicast (one-to-many)
E	240 – 255	—	Experimental / reserved

WARNING **Class A detail**

The range 127.0.0.0 – 127.255.255.255 is reserved for loopback (see Special addresses below). Therefore class A usable first octet is 1 to 126 only.

EXEMPLE **Reading an address**

The address `192.168.1.10` starts with 192, so it is class C. Default mask: `255.255.255.0`. Network part: `192.168.1`; host part: `10`. The network address is `192.168.1.0` and the broadcast address is `192.168.1.255`.

2.2 CIDR Notation

Modern networking uses **CIDR** (Classless Inter-Domain Routing) notation: a slash followed by the number of network bits. Example: `192.168.1.0/24` means the first 24 bits are the network prefix. This replaces classful boundaries and allows flexible prefix lengths (VLSM).

2.3 Private, Public and Special Addresses

Not every address may appear on the public Internet. The standard RFC 1918 reserves three ranges of **private addresses**, usable freely inside homes, schools and companies but never routed on the Internet:

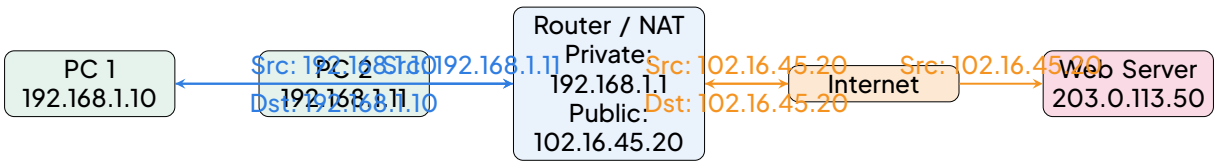
- `10.0.0.0` – `10.255.255.255` (class A, /8);
- `172.16.0.0` – `172.31.255.255` (class B, /12);
- `192.168.0.0` – `192.168.255.255` (class C, /16).

Public addresses are unique worldwide and allocated by Internet authorities to providers (in Cameroon, operators such as MTN, Orange or Camtel receive blocks for their customers).

KEY POINTS **Special addresses**

- Network address: host part all zeros (e.g. `192.168.1.0`) — identifies the network itself; never assigned to a machine.
- Broadcast address: host part all ones (e.g. `192.168.1.255`) — a packet sent here reaches all hosts of the network.
- Loopback `127.0.0.1`: a machine talking to itself, used to test the local TCP/IP stack (`ping 127.0.0.1`).
- Link-local (APIPA) `169.254.0.0/16`: self-assigned when DHCP fails.

Because private addresses cannot cross the Internet, a router performs **NAT** (Network Address Translation): it replaces the private source address of outgoing packets by its own public address, and reverses the translation for replies. NAT lets a whole cybercafé in Buea share a single public address — and it also hides the internal machines, a small security bonus.



NAT operation: private addresses are translated to a single public address for Internet traffic.

2.4 Subnetting: Dividing a Network

Intuition.

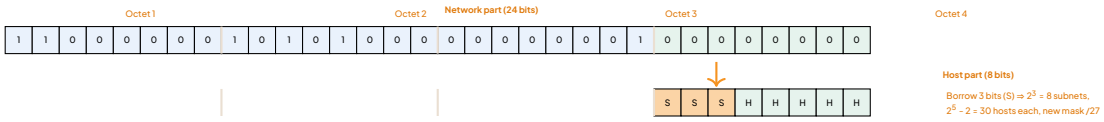
A school receives the network `192.168.10.0/24`: 254 usable addresses in one single block. But the school has a staff room, a computer lab and an administration office, and wants three separate networks for security and easier management. The solution is to borrow some host bits and use them as **subnet bits**.

If we borrow n bits from the host part:

- number of subnets created: 2^n;
- number of host addresses per subnet: $2^h - 2$, where h is the number of remaining host bits (we subtract the network and broadcast addresses).

METHOD **The 6-step subnetting procedure**

1. Identify the class and default mask from the first octet (or read the CIDR prefix).
2. Choose the number of bits to borrow n so that $2^n \geq$ required subnets (and enough hosts remain).
3. Write the new mask: default mask plus n ones; give it in dotted decimal and CIDR form (e.g. /27).
4. Compute the block size: 256- (value of the last modified mask octet). Subnet addresses are multiples of this block size.
5. List the subnets: network address of each subnet.
6. Write the host ranges: first usable = network +1; last usable = broadcast -1; broadcast = next subnet -1.



A class C address (`192.168.1.0`): the 24 network bits, the 8 host bits, and the effect of borrowing 3 bits for subnetting (mask becomes `255.255.255.224` = /27). S = subnet bits, H = remaining host bits.

WORKED EXAMPLE 2 **Subnetting a school network**

Statement: A school in Bamenda receives `192.168.10.0/24` and needs 5 subnets with as many hosts as possible in each. Give the new mask, the block size, and the first two subnets with their usable ranges.

SOLUTION

Solution (following the 6-step method):

1. First octet 192 ⇒ class C, default mask `255.255.255.0` (/24), 8 host bits.
2. We need $2^n \geq 5$, so $n = 3$ borrowed bits ($2^3 = 8$ subnets). Remaining host bits: $h = 8 - 3 = 5$, giving $2^5 - 2 = 30$ hosts per subnet.
3. New mask: 24 + 3 = 27 ones ⇒ /27, i.e. `255.255.255.224` (since 128 + 64 + 32 = 224).
4. Block size: 256 − 224 = 32. Subnets start at multiples of 32.
5. Subnets: `192.168.10.0`, `.32`, `.64`, `.96`, `.128`, `.160`, `.192`, `.224`.
6. First subnet: network `192.168.10.0`, usable `.1` – `.30`, broadcast `.31`. Second: network `.32`, usable `.33` – `.62`, broadcast `.63`.

WORKED EXAMPLE 3 **Subnetting a Class B network (VLSM introduction)**

Statement: A company has `172.16.0.0/16`. It needs three subnets: one for 500 hosts, one for 200 hosts, one for 50 hosts. Use Variable Length Subnet Masking (VLSM) to allocate efficiently.

SOLUTION

Solution:

1. Sort requirements descending: 500, 200, 50.
2. For 500 hosts: need $2^h - 2 \geq 500 \Rightarrow h = 10$ ($2^{10} - 2 = 1022$). Borrow 16 − 10 = 6 bits ⇒

mask /22. First subnet: `172.16.0.0/22` (range `172.16.0.0` – `172.16.3.255`).

3. Next free address: `172.16.4.0`. For 200 hosts: $h = 8$ ($2^8 - 2 = 254$). Borrow 8 bits ⇒ mask /24. Subnet: `172.16.4.0/24`.
4. Next free: `172.16.5.0`. For 50 hosts: $h = 6$ ($2^6 - 2 = 62$). Borrow 10 bits ⇒ mask /26. Subnet: `172.16.5.0/26`.

VLSM avoids wasting addresses by tailoring each subnet size to its need.

WARNING **Forgetting to subtract 2**

The most common exam mistake: writing $2^5 = 32$ hosts per subnet. Two addresses are never assignable: the network address (all host bits 0) and the broadcast address (all host bits 1). Usable hosts = $2^h - 2$. Likewise, `192.168.10.32` in the example above is a network address, not a machine address.

A word on IPv6.

With about $2^{32} \approx 4.3$ billion addresses, IPv4 is exhausted: there are more connected devices than addresses. **IPv6** uses **128-bit** addresses written in hexadecimal, e.g. `2001:0db8:85a3::8a2e:0370:733` providing an effectively unlimited address space and built-in security features (IPsec mandatory). NAT was only a temporary workaround; IPv6 is the long-term solution. IPv6 addresses are written as eight groups of four hex digits; leading zeros in a group may be omitted, and one sequence of all-zero groups may be replaced by `::`.

2.5 Security Goals: The CIA Triad

Connecting machines creates value — and risk. Network security is organised around five goals, the first three forming the famous **CIA triad**:

Pillar	Meaning	Cameroonian example
Confidentiality	Only authorised persons can read the data.	A student's GCE results must be readable only by the candidate and the Board, not by neighbours.
Integrity	Data cannot be altered without detection.	The balance of an MTN Mobile Money account must not be modifiable in transit.
Availability	Services and data are accessible when needed.	A university's online registration portal must stay reachable during registration week.

The CIA triad, with one everyday Cameroonian example per pillar.

Two further goals complete the picture: **authentication** (being sure of who is at the other end) and **non-repudiation** (a sender cannot later deny having sent a message or made a transaction — essential for mobile money disputes).

2.6 Threats and Attacks

Malicious software (malware).

- **Virus**: a program that attaches itself to a legitimate file and spreads when that file is executed or shared (e.g. via an infected USB flash drive — very common in school computer labs).
- **Worm**: self-replicating without attaching to a file; it spreads by itself across the network exploiting vulnerabilities.
- **Trojan horse**: hides inside an apparently useful program (a "free" game or cracked software) and opens a backdoor for the attacker.
- **Ransomware**: encrypts the victim's files and demands payment for the decryption key.
- **Spyware**: secretly monitors the user (keystrokes, passwords, browsing) and sends the data to an attacker.

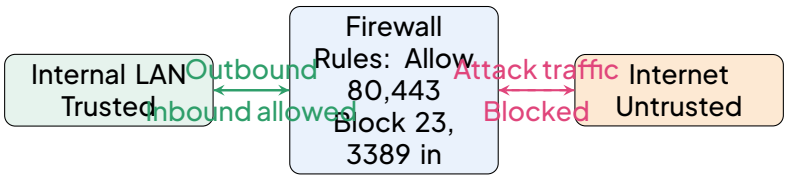
Human and network attacks.

- **Phishing / social engineering**: tricking the person, not the machine — e.g. an SMS pretending to come from a mobile money service asking for your PIN.
- **Interception (sniffing)**: capturing packets travelling on a network to read passwords or messages sent in clear text (e.g. on unencrypted HTTP or Telnet).
- **DoS / DDoS** (Denial of Service): flooding a server with requests until it collapses; distributed when thousands of hijacked machines (botnet) attack together.
- **Man-in-the-Middle (MitM)**: attacker secretly relays and possibly alters communication between two parties who believe they are directly connected.
- **Unauthorised access**: guessing, stealing or cracking passwords to enter accounts or systems.
- **SQL Injection**: inserting malicious SQL code into input fields to manipulate a database.

2.7 Countermeasures

DEFINITION **Firewall**

A **firewall** is a hardware or software barrier placed between a network and the outside world that filters incoming and outgoing traffic according to security rules (allowed ports, addresses, protocols), blocking unauthorised connections while letting legitimate traffic pass.



Firewall filtering between trusted LAN and untrusted Internet.

A complete defence combines several layers (defence in depth):

- **Firewall** at the network border and on each machine (host-based firewall);
- **Antivirus / Anti-malware** software, kept up to date, to detect and remove malware;
- **Access control and strong passwords**: long passwords mixing letters, digits and symbols; each user has only the rights needed (principle of least privilege);
- **Regular backups** on separate media (offline or off-site) — the only reliable answer to ransomware;
- **Encryption** for data in transit (TLS/HTTPS, SSH, VPN) and at rest (disk encryption);
- **Physical security**: locked server rooms, controlled access to equipment;
- **User awareness**: training users never to share PINs, to verify senders, and to distrust "urgent" messages.
- **Updates and patch management**: keep OS, firmware and applications patched against known vulnerabilities.

WARNING **The weakest link is the user**

The best firewall in the world is useless if a secretary writes the server password on a paper stuck to the monitor, or if a student clicks a link promising free airtime. Most successful attacks exploit people, not technology. Technical controls must always be accompanied by awareness training and clear security rules.

2.8 Cryptography: Protecting Data with Mathematics

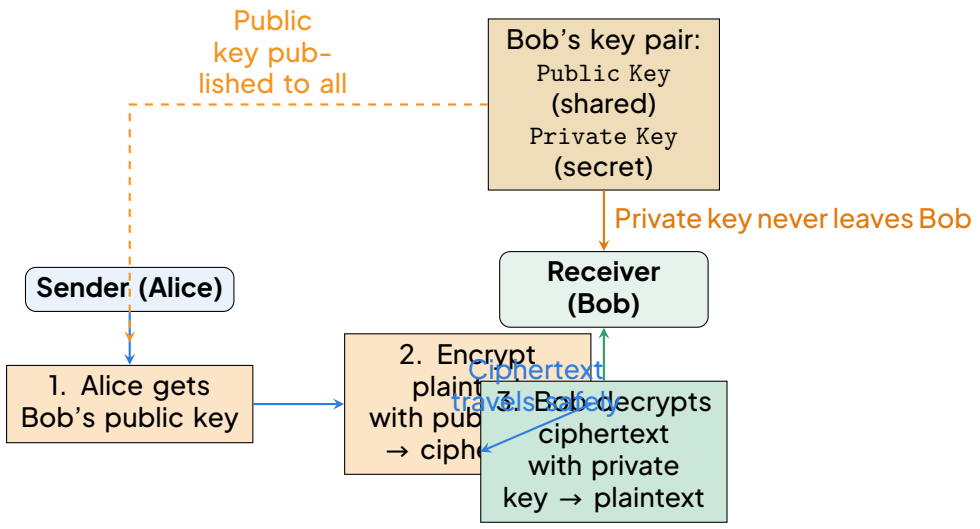
DEFINITION **Encryption**

Encryption is the transformation of a readable message (plaintext) into an unreadable form (ciphertext) using an algorithm and a key, so that only someone possessing the correct key can reverse the operation (decryption).

There are two great families:

PROPERTY **Symmetric vs asymmetric cryptography**

Symmetric (secret-key): the same key encrypts and decrypts. Very fast, ideal for large amounts of data — but the key must be shared secretly in advance, which is the key distribution problem. Examples: AES, DES, ChaCha20.
Asymmetric (public-key): each user has a public key (shared with everyone, used to encrypt) and a private key (kept secret, used to decrypt). Solves key distribution and enables digital signatures, but is much slower. Examples: RSA, ECC (Elliptic Curve Cryptography).
In practice, systems like HTTPS combine both: asymmetric cryptography exchanges a temporary symmetric key (session key), which then encrypts the bulk of the communication. (The comparison itself is examinable; mathematical proofs of the algorithms are not.)



Public-key encryption: anyone can encrypt with Bob's public key, but only Bob's private key can decrypt.

Two applications follow directly. A **digital signature** is created by hashing the message and encrypting the hash with the sender's private key: anyone can verify it with the public key, which proves who signed (authentication), that the message was not modified (integrity), and provides non-repudiation. A **digital certificate**, issued by a trusted **Certificate Authority (CA)**, binds a public key to an identity — this is what your browser checks when it displays the padlock of **HTTPS** (HTTP secured by SSL/TLS). Every time a user in Douala confirms a mobile money payment or checks a bank account online, asymmetric cryptography, certificates and symmetric session keys are working together behind the scenes.

2.9 Authentication and Legal Aspects

To prove an identity, systems rely on three factor types:

- **Something you know**: password, PIN, pattern;
- **Something you have**: SIM card, bank card, hardware token (OTP generator), smartphone with authenticator app;
- **Something you are**: fingerprint, face recognition, iris scan (biometrics).

Two-factor authentication (2FA) combines two different types — e.g. a password plus a code sent by SMS or generated by an app — so that stealing one factor is not enough. This is why mobile money services require both possession of the SIM and the secret PIN. **Multi-factor authentication (MFA)** uses two or more factors.

Finally, security is also a matter of **law and ethics**. Cameroon has legislation punishing cybercrime (Law No. 2010/012 of 21 December 2010 on cybersecurity and cybercrime) — unauthorised access to systems, data theft, online fraud and identity theft are criminal offences, and the country cooperates internationally against cybercrime (Budapest Convention). Beyond the law, an ethical attitude is expected of every computer scientist: never access systems without permission, never disclose others' data, and use skills to protect rather than to harm.

EXERCISE 2 **Subnetting and security**

1. A company receives `172.16.0.0/16`. How many usable hosts does the unsubnetted network provide?
2. Subnet `192.168.5.0/24` into 4 equal subnets. Give the mask, block size, and the usable range of the third subnet.
3. Classify each threat: (a) a program that replicates alone across the network; (b) an SMS asking for your mobile money PIN; (c) a flood of requests crashing a website; (d) an attacker intercepting and modifying traffic between two parties.
4. Explain why HTTPS uses both symmetric and asymmetric encryption.
5. A network administrator configures a firewall to block all incoming traffic on port 23 (Telnet) but allow port 22 (SSH). Explain the security reasoning.

SOLUTION **Exercise 1**

1. Class B, 16 host bits: $2^{16} - 2 = 65\,534$ usable hosts.
2. Borrow 2 bits ($2^2 = 4$): mask /26 = `255.255.255.192`; block size 256−192 = 64. Subnets: `.0`, `.64`, `.128`, `.192`. Third subnet: network `192.168.5.128`, usable `.129` – `.190`, broadcast `.191`.
3. (a) worm; (b) phishing / social engineering; (c) DoS (DDoS if distributed); (d) Man-in-the-Middle (MitM).

4. Asymmetric encryption safely exchanges a secret session key between strangers; the fast symmetric algorithm then encrypts the actual data. Asymmetric alone would be too slow for whole pages and transactions.
5. Telnet sends data (including passwords) in clear text, making it vulnerable to sniffing. SSH encrypts the session, providing confidentiality and integrity. Blocking Telnet forces the use of the secure alternative.

KEY POINTS **Key points**

- IPv4 = 32 bits, dotted decimal; classes A/B/C with masks /8, /16, /24; private ranges RFC 1918; NAT shares one public address.
- CIDR notation: `address/prefix-length` (e.g., `192.168.1.0/24`).
- Subnetting: borrow n bits ⇒ 2^n subnets, $2^h - 2$ usable hosts; never forget network and broadcast addresses.
- VLSM allows different subnet sizes for different needs, optimising address space.
- IPv6 = 128 bits, hex notation, huge address space, IPsec built-in.
- Security goals: confidentiality, integrity, availability, authentication, non-repudiation (CIA + A + NR).
- Threats: malware (virus, worm, trojan, ransomware, spyware), phishing, sniffing, DoS/DDoS, MitM, SQL injection.
- Defence in depth: firewall, antivirus, strong passwords, least privilege, backups, encryption, patching, physical security, user awareness.
- Symmetric = fast, shared key; asymmetric = public/private pair, signatures and certificates; HTTPS combines both; 2FA/MFA combines factor types.
- Legal framework: Cameroon Law 2010/012 on cybersecurity and cybercrime; ethical responsibility of IT professionals.

3 SOFTWARE ENGINEERING, SDLC & THEORETICAL COMPUTER SCIENCE

In the previous section we saw how computers exchange data across networks and how we protect that data. But every network, every server and every security tool runs software — and software, unlike hardware, is not manufactured in a factory: it is designed, written, tested and maintained by human beings. Experience has shown that writing software without a disciplined

method leads to late, over-budget and unreliable systems. This section presents the discipline that was invented to solve that problem — **software engineering** — and then opens a window onto the theoretical foundations of computing itself: what can be computed, how fast, and how programs are translated.

3.1 Software Engineering: Definition and Goals

EXPERIMENT **The software crisis**

In the late 1960s, as computers became more powerful, software projects grew enormously — and many failed spectacularly. Projects were delivered years late, cost several times their budget, or were abandoned altogether. The term software crisis was popularised at the 1968 NATO Software Engineering Conference in Garmisch, Germany. The diagnosis was simple: programmers were using the informal habits that work for small programs on projects involving millions of lines of code. The remedy proposed was to treat software development as an engineering discipline, like building a bridge: analyse, design, build, test, maintain — with documentation at every stage.

DEFINITION **Software engineering**

Software engineering is the systematic application of engineering principles, methods and tools to the development, operation and maintenance of software, so that it is produced on time, within budget and with the required quality.

The three classical goals of software engineering are therefore:

- **Quality**: the software must be correct (does what is specified), reliable (fails rarely), efficient, usable and maintainable.
- **Cost**: development must stay within the planned budget of money and human effort.
- **Schedule**: the product must be delivered by the agreed deadline.

These three goals are in permanent tension: increasing quality usually costs more time and money. Managing this trade-off is the daily work of a software engineer.

3.2 The Software Development Life Cycle (SDLC)

DEFINITION **SDLC**

The **Software Development Life Cycle** (SDLC) is the structured sequence of phases through which a software product passes, from the initial idea to its final retirement.

The standard phases are:

1. **Feasibility study**: can the project be done? We examine technical feasibility (do we have the technology and expertise?), economic feasibility (cost-benefit analysis: is it worth the cost?), legal/organisational feasibility (compliance with laws, company policies) and schedule feasibility (can it be done in time?).
2. **Requirements analysis**: what must the system do? Analysts interview users and stakeholders, observe current workflows, and record needs — often producing a preliminary requirements document.
3. **Specification**: the requirements are written down precisely and formally in a Software Requirements Specification (SRS), which becomes the contract between customer and developer. The SRS must be complete, consistent, unambiguous and verifiable.
4. **Design**: how will the system do it? This phase produces the software architecture (high-level design: modules, data flow, interfaces, database schema, technology stack) and detailed design (algorithms, data structures, class diagrams, UI mock-ups) — before any code is written.
5. **Implementation (coding)**: programmers translate the design into source code in a chosen programming language, following coding standards and using version control.
6. **Testing**: the software is checked systematically against the specification (see Testing subsection below).
7. **Deployment**: the software is installed in the real environment (production), data is migrated, users are trained, and a cut-over plan is executed.
8. **Maintenance**: faults are corrected and the software is adapted and improved throughout its working life. Maintenance is usually the longest and most expensive phase of all (often 60–80% of total lifecycle cost).

> **WARNING** **Design before code**
>
> A very common beginner's mistake is to start coding immediately. In engineering, a fault discovered during maintenance can cost 10 to 100 times more to repair than the same fault caught during design. The earlier an error is found, the cheaper it is to fix.

3.3 Life-Cycle Models Compared

A **life-cycle model** is a particular way of organising the SDLC phases. The choice of model depends on the project context.

The waterfall model. Each phase is completed fully before the next begins; the process flows downwards like a waterfall, with limited feedback to the previous phase. It is simple to manage and suits projects with stable, well-understood requirements (e.g., a payroll system with fixed rules). Its weakness: a mistake in the requirements is discovered only at testing, when it is very costly.

The V-model. A variant of the waterfall in which every development phase on the left of the "V" has a corresponding test phase on the right: Requirements ↔ Acceptance Testing, Architec-

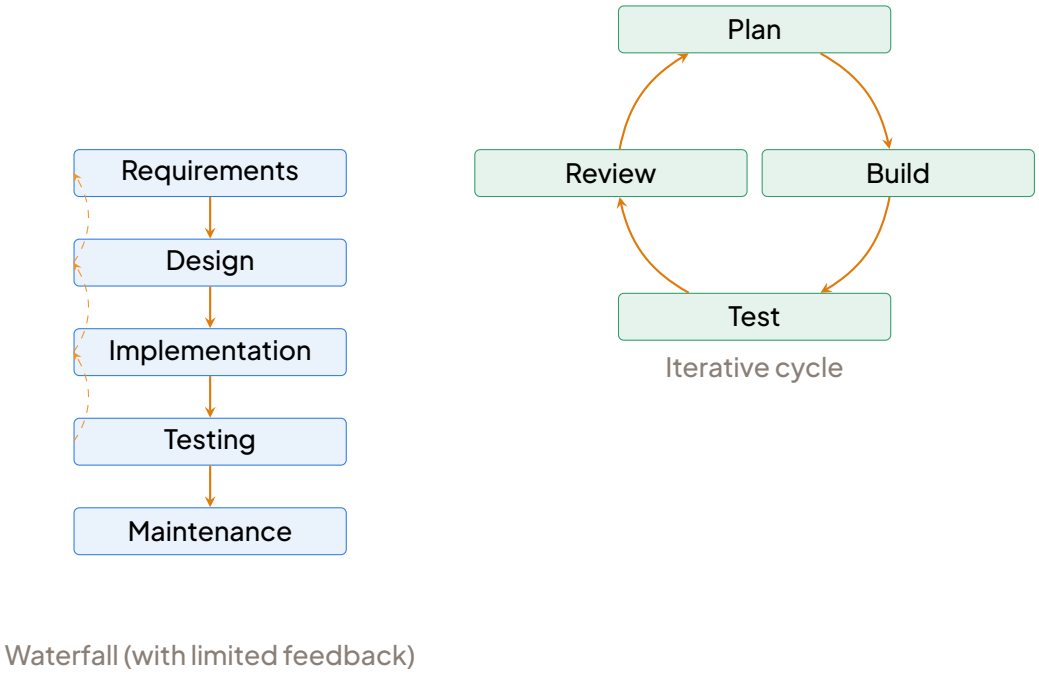
tural Design ↔ System Testing, Detailed Design ↔ Integration Testing, Coding ↔ Unit Testing. It emphasises early test planning and suits safety-critical systems where verification is paramount.

Iterative and incremental models. The product is built in repeated cycles (iterations); each cycle adds working functionality (increments) and feedback from one cycle improves the next. This reduces risk because a working version exists early and requirements can be refined.

Prototyping. A quick, incomplete mock-up of the system (the prototype) is built to clarify requirements with the customer, then the real system is developed (often using a different model). Ideal when users "don't know what they want until they see it".

The spiral model. The project is repeated through four quadrants — Determine Objectives, Identify and Resolve Risks, Develop and Test, Plan Next Iteration — each loop producing a more complete version. It is risk-driven and suits large, expensive, high-risk projects (e.g., satellite control systems).

Agile methods (Scrum basics). Agile development values individuals and interactions over processes and tools, working software over comprehensive documentation, customer collaboration over contract negotiation, and responding to change over following a plan. In **Scrum**, work is divided into short fixed-length iterations called sprints (typically 2–4 weeks); a small self-organising team picks items from a prioritised list (the product backlog), meets briefly every day (the daily stand-up), and delivers a potentially usable product increment at the end of each sprint.



The waterfall model (left) flows through each phase once; the iterative model (right) repeats a short cycle, improving the product each time.

PROPERTY **Waterfall vs Agile — Comparison for Examination**

Criterion	Waterfall (or V-model)	Agile (e.g., Scrum)
Requirements	Fixed, fully specified at the start	Expected to evolve; re-prioritised each sprint
Customer involvement	Mainly at the beginning and the end	Continuous; customer reviews every increment
Risk handling	Risks appear late (at testing)	Risks surfaced early by frequent deliveries
Documentation	Heavy and contractual	Light; working software preferred
Best for	Stable, well-understood, regulated projects	Changing requirements, innovative products

(Comparison to know; justification in a scenario is examinable.)

WARNING **Exam focus — GCE**

A typical GCE question describes a project scenario and asks you to choose and justify a life-cycle model. Method: identify (1) whether requirements are stable or changing, (2) how available the customer is, (3) the size and risk of the project. Stable requirements + fixed contract ⇒ waterfall/V-model; unclear requirements ⇒ prototyping; high risk and large budget ⇒ spiral; changing requirements and an available customer ⇒ agile.

3.4 Requirements and Design Tools

Functional vs non-functional requirements. A **functional requirement** states what the system must do: "the system shall print a report card for each student". A **non-functional requirement** states how well it must do it: performance ("respond within 2 seconds"), security ("encrypt all data at rest"), usability, reliability ("99.9% uptime"), portability, maintainability.

Design tools every Upper Sixth candidate must recognise and be able to interpret:

- **Use case diagram**: shows the actors (users or external systems) and the services (use cases) the system offers them — a view of the system from outside.
- **Data flow diagram (DFD)**: shows how data moves between processes (bubbles), data stores (open rectangles) and external entities (rectangles), without saying how the processes work. Level 0 (context diagram) shows the whole system as one process.
- **Flowchart**: represents the logic of an algorithm with standard symbols (oval = start/stop, parallelogram = input/output, rectangle = process, diamond = decision, arrow = flow).
- **Pseudocode**: an informal, language-independent description of an algorithm.

- **Structure chart**: a hierarchical diagram showing how a program is decomposed into modules and which module calls which (top-down decomposition).

EXEMPLE **Pseudocode example**

The following pseudocode reads a mark and outputs the grade.

```
INPUT mark
IF mark < 0 OR mark > 100 THEN
    OUTPUT "Invalid mark"
ELSE IF mark >= 80 THEN
    OUTPUT "Distinction"
ELSE IF mark >= 50 THEN
    OUTPUT "Pass"
ELSE
    OUTPUT "Fail"
ENDIF
```

3.5 Testing

DEFINITION **Verification and validation**

Verification asks: "Are we building the product right?" — does each phase's output conform to its specification (reviews, inspections, walkthroughs, unit tests)? **Validation** asks: "Are we building the right product?" — does the final system meet the real needs of the user (acceptance testing with the customer)?

WARNING **Do not confuse them**

Verification is an internal, technical check against documents; validation is an external check against user needs. A program can pass verification (it matches the specification perfectly) and still fail validation (the specification itself was wrong).

Levels of testing, from smallest to largest scope:

1. **Unit testing**: each module or function is tested in isolation, usually by the programmer (white-box).
2. **Integration testing**: modules are combined and the interfaces between them are tested (top-down, bottom-up, or big-bang).

3. **System testing**: the complete, integrated system is tested against the specification, including non-functional aspects (performance, security, stress, usability).
4. **Acceptance testing**: the customer checks that the system satisfies their needs before accepting delivery (sometimes split into alpha testing at the developer's site and beta testing by selected real users in their own environment).
5. **Regression testing** (during maintenance): re-running tests after a change to ensure no new faults were introduced.

Black-box vs white-box testing. In **black-box** testing, the tester ignores the internal code and chooses test data from the specification (inputs and expected outputs). Techniques: equivalence partitioning, boundary value analysis. In **white-box** testing, the tester knows the code and designs tests to exercise every statement (statement coverage) and every branch (branch/decision coverage).

METHOD **Deriving test data: normal, boundary, erroneous**

Given a specification with defined input ranges, produce three classes of test data:

1. Normal (typical) cases: ordinary valid values well inside the allowed range.
2. Boundary (extreme) cases: values exactly on, and immediately either side of, the limits of valid ranges — most bugs hide at boundaries.
3. Erroneous (invalid) cases: values outside the allowed range or of the wrong type, to check that the program rejects them gracefully.

Record each test in a test plan: test number, input data, expected result, actual result, pass/fail.

WORKED EXAMPLE 4 **Test data for a grading program**

Statement. A program accepts an integer mark and must display "Invalid" if the mark is outside 0–100, "Distinction" for 80–100, "Pass" for 50–79 and "Fail" for 0–49. Propose test data covering all classes.

SOLUTION

Solution.

Type	Input	Expected	Reason
Normal	65	Pass	Typical mid-range value
Boundary	49, 50	Fail, Pass	Either side of the Pass limit
Boundary	79, 80	Pass, Distinction	Either side of the Distinction limit
Boundary	0, 100	Fail, Distinction	Ends of the valid range
Erroneous	-1, 101	Invalid, Invalid	Just outside the range
Erroneous	“abc”	Invalid	Wrong data type

Notice how the boundaries 49/50 and 79/80 are where a programmer writing `mark > 50` instead of `mark >= 50` would be caught.

3.6 Documentation and Maintenance

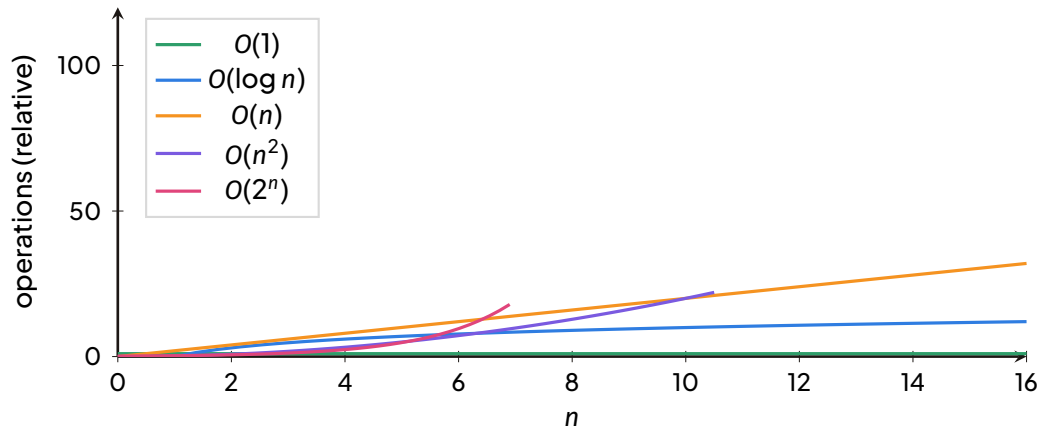
Documentation comes in two families. **User documentation** (user manual, online help, installation guide, tutorial) explains to end-users how to operate the software. **Technical documentation** (SRS, design documents, data dictionary, commented source code, test plans, test logs, maintenance manuals) is written for the developers who will modify the system later.

Types of maintenance (a classic examination list — know all four):

- **Corrective**: repairing faults discovered after delivery (bug fixes).
- **Adaptive**: modifying the software so it keeps working when its environment changes (new operating system, new hardware, new tax law, new database version).
- **Perfective**: improving performance, maintainability or adding features requested by users (enhancements).
- **Preventive**: restructuring or re-documenting the code now (refactoring) to make future faults and changes less likely.

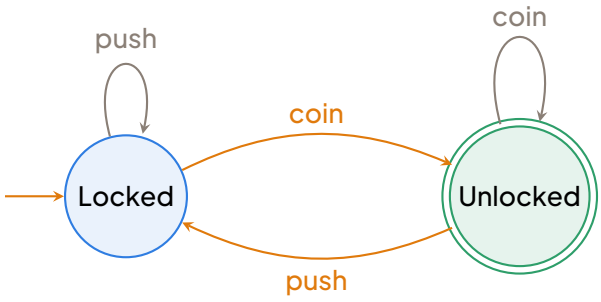
3.7 Theoretical Computer Science: An Overview

Algorithms and complexity. The efficiency of an algorithm is measured by how its running time (or memory) grows with the input size n. We express this growth with **big-O notation**, which keeps only the dominant term for large n: searching a sorted list by binary search is $O(\log n)$; scanning a list once is $O(n)$; efficient sorting (merge sort, quick sort) is $O(n \log n)$; comparing every pair of items (bubble/selection sort) is $O(n^2)$; trying all subsets (travelling salesman by brute force) is $O(2^n)$. Problems solvable in polynomial time ($O(n)$, $O(n^2)$, $O(n^3)$, …) are called **tractable**; problems for which the best known algorithms are exponential, like $O(2^n)$ or $O(n!)$, are **intractable** for large n — doubling n does not double the work, it squares it or worse.



Growth of common complexity classes: constant and logarithmic algorithms scale beautifully; exponential growth explodes even for small n.

Finite automata. A **finite-state automaton** (FSA) is a mathematical model of a machine that is always in one of a finite number of states and changes state when it receives an input, according to transitions. It is drawn as a state transition diagram: circles for states (a double circle for an accepting/final state), arrows for transitions labelled by the input symbol. A deterministic FSA (DFA) has exactly one transition per state per input symbol. FSAs model vending machines, turnstiles, protocol logic (TCP handshake) and the lexical analysers inside compilers.



Deterministic finite automaton for a turnstile: inserting a coin unlocks it; pushing it through locks it again. Extra coins or pushes in the wrong state change nothing (self-loops).

Computability and the halting problem. Not every well-stated problem can be solved by any algorithm. Alan Turing proved in 1936 that there is no general program that can examine any other program P and input x and always decide whether $P(x)$ eventually stops or loops forever — this is the **halting problem**. Intuitively: if such a "halt-checker" H existed, one could build a program D that takes a program P as input, runs $H(P, P)$, and does the opposite of what H predicts (halts if H says loops, loops if H says halts). Running $D(D)$ produces a contradiction. The lesson is profound: computers have fundamental limits, independent of speed or memory.

Compilers vs interpreters. A **compiler** translates the whole source program into machine code (or bytecode) once, producing an executable that then runs quickly; errors are reported at trans-

lation time. An **interpreter** translates and executes the source line by line (or statement by statement) each time the program runs; this is slower but excellent for learning and debugging, since execution stops exactly at the offending line. (Many modern languages, such as Java and Python, use a hybrid: compile to bytecode, then interpret or just-in-time compile the bytecode to native code.)

3.8 Professional and Ethical Issues

Software engineers have legal and moral responsibilities:

- **Intellectual property**: software is protected by copyright; copying, distributing or modifying a program without permission is piracy, punishable by law in Cameroon (Law No. 2000/011 on Copyright) and internationally.
- **Software licences**: a **proprietary licence** (e.g., typical commercial software) grants the right to use the program but not to see or modify its source code. An **open-source licence** (e.g., GNU GPL, MIT, Apache) makes the source code available and permits study, modification and redistribution under stated conditions. "Free" in open source refers to freedom, not necessarily price.
- **Reliability and safety-critical systems**: software that controls aircraft, medical devices, banking systems or electricity grids can kill or ruin people if it fails. Such safety-critical software demands the most rigorous methods: formal specification, the V-model, exhaustive testing and independent verification.
- **Professional conduct**: honesty about capabilities and deadlines, respect for user privacy and data protection (Cameroon Law No. 2010/012 on Cybersecurity and Cybercrime), and refusal to build systems designed to deceive or harm.

DID YOU KNOW? **Open source in daily life**

The Android phones used all over Cameroon run on the Linux kernel, which is open source; so do most web servers on the Internet. Students can legally download, study and improve such software — an extraordinary free learning resource.

EXERCISE 3 **Choosing a life-cycle model**

A start-up in Douala wants a mobile application that connects farmers to buyers. The founders admit their ideas "will evolve as users react", and they can meet the developers every two weeks. A bank also asks the same team for a system calculating loan interest according to fixed, published rules. Recommend a life-cycle model for each project and justify.

SOLUTION **Exercise 1**

For the farmers' application: an agile approach (e.g., Scrum). Justification: requirements are unstable and expected to change; the customers are available for regular feedback (sprint reviews); early working versions reduce the risk of building the wrong product. For the bank's interest calculator: the waterfall (or V-) model. Justification: requirements are fixed, precise and rule-based; correctness is critical and must be verified against a stable specification; there is little benefit from iteration.

KEY POINTS **Key points for revision**

- Software engineering exists because of the software crisis; its goals are quality, cost control and schedule.
- SDLC: feasibility, requirements, specification, design, implementation, testing, deployment, maintenance.
- Choose the life-cycle model from the scenario: stable requirements → waterfall/V; unclear → prototyping; high risk → spiral; changing + available customer → agile.
- Verification = building the product right; validation = building the right product. Testing levels: unit, integration, system, acceptance. Test data: normal, boundary, erroneous.
- Maintenance: corrective, adaptive, perfective, preventive.
- Big-O measures growth; polynomial problems are tractable, exponential ones are intractable; the halting problem shows some problems are undecidable; compilers translate once, interpreters line by line.
- Ethics: copyright, licences (proprietary vs open source), safety-critical rigour, data protection.

4 HANDS-ON NETWORKING LAB: SUBNETTING & LAN SIMULATION

In the previous section we studied software engineering and the SDLC: we saw that any serious computing project follows a disciplined cycle of **analysis, design, implementation, testing and maintenance**. In this final section we put that philosophy into practice on the networking concepts of Sections 1 and 2. We shall treat the construction of a small school network exactly like a mini software project: we plan the addressing on paper, we implement it in a simulator, we test it systematically, and we document it in a short technical report. This is precisely the kind of practical, structured work the GCE Advanced Level expects you to describe and

analyse.

4.1 The Lab Environment: Cisco Packet Tracer

Real networking hardware is expensive, so engineers learn and prototype using **network simulators**. The most widely used in schools is **Cisco Packet Tracer**, a free simulator that lets you drag routers, switches and PCs onto a workspace, cable them, configure them through realistic command-line interfaces, and observe packets moving across the topology in slow motion. Equivalent tools exist (GNS3, EVE-NG, or even physical equipment in a school lab), and everything in this section applies to them too.

The Packet Tracer workspace has three essential areas:

- The **workspace** (centre), where devices are placed and connected.
- The **device palette** (bottom-left), organised by category: routers, switches, end devices (PCs, servers, printers), and connections (cables).
- The **configuration window**, opened by clicking a device: a PC offers a Desktop tab (IP configuration, command prompt, web browser), while a router or switch offers a CLI tab (command-line interface) and a Config tab (GUI configuration).

Choosing the correct **cable** is the first practical skill:

- A **straight-through** cable (copper straight-through) connects different types of device: PC to switch, switch to router, PC to hub.
- A **crossover** cable connects similar devices: PC to PC, switch to switch, router to router (and, by convention, PC directly to router).
- A **console** cable (light blue, rolled cable) connects a PC's serial port (or USB-to-serial adapter) to a router's console port for initial out-of-band configuration.
- A **serial** cable (DCE/DTE) connects two routers back-to-back for WAN links in the lab.

WARNING **Wrong cable, dead link**

If a link shows red dots at both ends in Packet Tracer, the most common cause at the start is a wrong cable type or a shutdown interface. Modern equipment supports auto-MDIX (automatic crossover detection), but in the exam and in the simulator you should know the classical rules above. Always wait for the link lights to turn green before configuring IP addresses.

DID YOU KNOW? **Packet Tracer Simulation Mode**

Beyond Realtime mode, Packet Tracer offers Simulation Mode (Shift+S). This lets you capture and inspect every packet (ARP requests, ICMP echoes, DNS queries, TCP handshakes) step by step. Use it to see the OSI layers in action: Layer 2 MAC addresses, Layer 3 IP addresses, Layer 4 port numbers. This is invaluable for understanding encapsulation and for

the GCE practical questions on protocol analysis.

4.2 Guided Subnetting Workshop: 192.168.10.0/24

Scenario. A school in Bamenda has four departments sharing the private network `192.168.10.0/24`: Administration, Science, Arts, and the Computer Laboratory. Each department must be in its own subnet for security and manageability. We must produce at least four subnets.

Paper work first — the subnetting algorithm.

1. **Count required subnets:** 4 departments ⇒ need ≥ 4 subnets.
2. **Find bits to borrow:** smallest n such that $2^n \geq 4 \Rightarrow n = 2$ bits.
3. **New prefix length:** original /24 + 2 = /26.
4. **New subnet mask:** write 26 ones then 6 zeros:

$$\underbrace{11111111.11111111.11111111}_{24\text{ bits}}\ \underbrace{11}_{2\text{ borrowed}}\ 000000 = 255.255.255.192$$

5. **Block size (magic number):** $2^{32-26} = 2^6 = 64$ addresses per subnet.
6. **Usable hosts per subnet:** 64 − 2 = 62 (subtract network and broadcast addresses).
7. **List subnets:** start at 0, increment by 64 in the last octet: 0, 64, 128, 192.

Subnet	Network address	Usable host range	Broadcast address	Mask
Administration	192.168.10.0/26	192.168.10.1 – 192.168.10.62	192.168.10.63	255.255.255.192
Science	192.168.10.64/26	192.168.10.65 – 192.168.10.126	192.168.10.127	255.255.255.192
Arts	192.168.10.128/26	192.168.10.129 – 192.168.10.190	192.168.10.191	255.255.255.192
Computer Lab	192.168.10.192/26	192.168.10.193 – 192.168.10.254	192.168.10.255	255.255.255.192

WARNING **Network and broadcast addresses are not host addresses**

A classic exam and lab error is to assign the network address (e.g. `192.168.10.64`) or the broadcast address (e.g. `192.168.10.127`) to a PC or router interface. The network address identifies the subnet itself; the broadcast address targets all hosts of the subnet. Neither may identify a single interface. Only addresses strictly inside the usable range are valid.

KEY POINTS **Powers of two for the exam**

Paper-based subnetting questions are extremely frequent at the GCE. Learn by heart the powers of two up to $2^{10} = 1024$ and the mask values produced by borrowing bits in the last octet:

Bits borrowed	New mask (last octet)	Block size
1	128 (/25)	128
2	192 (/26)	64
3	224 (/27)	32
4	240 (/28)	16
5	248 (/29)	8
6	252 (/30)	4
7	254 (/31)	2 (point-to-point links)
8	255 (/32)	1 (single host)

With these, any /24 subnetting exercise can be done in under two minutes without a calculator.

4.3 Variable Length Subnet Mask (VLSM) — Extension for the Exam

The GCE may ask you to subnet a network into subnets of different sizes (e.g. one subnet for 60 hosts, one for 25, two for point-to-point links). This is **VLSM**. The rule: **start with the largest subnet first**, then carve the remaining space for smaller ones.

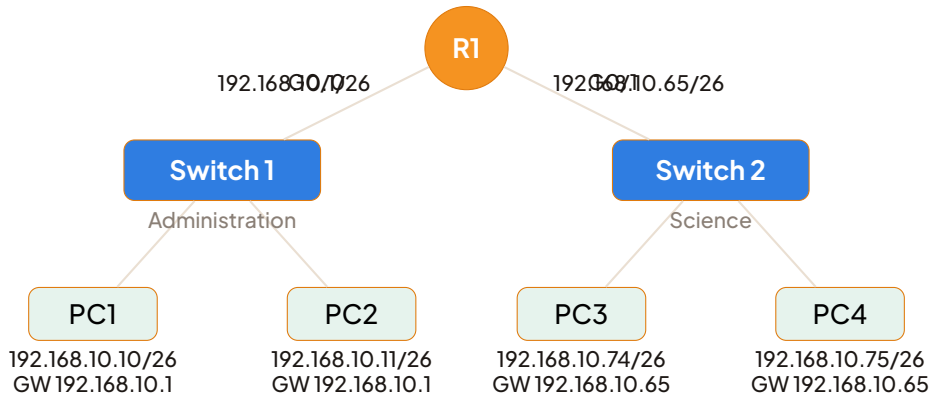
EXEMPLE **VLSM Example: 192.168.10.0/24 for 60, 25, 2, 2 hosts**

1. 60 hosts ⇒ need 62 usable ⇒ /26 (block 64). Subnet: `192.168.10.0/26` (0–63).
2. 25 hosts ⇒ need 27 usable ⇒ /27 (block 32). Next free block: `192.168.10.64/27` (64–95).
3. 2 hosts (p2p) ⇒ /30 (block 4). Next free: `192.168.10.96/30` (96–99).
4. 2 hosts (p2p) ⇒ /30. Next free: `192.168.10.100/30` (100–103).

Remaining space (104–255) is reserved for growth. No overlaps — this is the key check.

4.4 Building the Topology

To keep the lab readable while remaining realistic, we implement two of the four subnets (Administration and Science) with one router, two switches and four PCs. The router's two interfaces are the **default gateways** of the two subnets.



Reference lab topology: one router, two switches, four PCs across two /26 subnets. Router interfaces act as default gateways.

Step-by-step configuration:

1. **PC IP configuration** (Desktop → IP Configuration → Static):
   - PC1: IP `192.168.10.10`, Mask `255.255.255.192`, Gateway `192.168.10.1`
   - PC2: IP `192.168.10.11`, Mask `255.255.255.192`, Gateway `192.168.10.1`
   - PC3: IP `192.168.10.74`, Mask `255.255.255.192`, Gateway `192.168.10.65`
   - PC4: IP `192.168.10.75`, Mask `255.255.255.192`, Gateway `192.168.10.65`
2. **Router CLI configuration** (CLI tab):

```
Router> enable
Router\# configure terminal
Router(config)\# hostname R1
R1(config)\# interface gigabitEthernet 0/0
R1(config-if)\# ip address 192.168.10.1 255.255.255.192
R1(config-if)\# no shutdown
R1(config-if)\# exit
R1(config)\# interface gigabitEthernet 0/1
R1(config-if)\# ip address 192.168.10.65 255.255.255.192
R1(config-if)\# no shutdown
R1(config-if)\# end
R1\# write memory
```

WARNING **The `no shutdown` command**

Router interfaces are administratively down by default (unlike switch ports which are up). Forgetting `no shutdown` is the number one reason a correctly addressed lab fails. Always verify with `show ip interface brief` — the Status and Protocol columns must both read `up`.

4.5 Static Routing and the Routing Table

A router knows directly only the networks physically connected to its interfaces. To reach remote networks it needs **routes**. With **static routing**, the administrator enters each route by hand:

- `ip route <network> <mask> <next-hop-IP>` (next-hop = neighbour router's interface)
- `ip route <network> <mask> <exit-interface>` (alternative for point-to-point links)

Example: If a second router R2 (connected via `192.168.10.129/30` on a serial link) served the Arts subnet (`192.168.10.128/26`), R1 would need:

```
R1(config)\# ip route 192.168.10.128 255.255.255.192 192.168.10.130
```

(where `192.168.10.130` is R2's serial interface IP).

The command `show ip route` displays the **routing table**. Key codes:

- `C` — directly **Connected** networks (learned automatically from active interfaces).
- `S` — **Static** routes (manually configured).
- `S*` — candidate **default route** (gateway of last resort).
- `O, D, R` — dynamic routing protocols (OSPF, EIGRP, RIP) — not required for this syllabus but good to recognise.

In our single-router lab both subnets are connected, so no static route is needed between them — but the concept is examinable and essential as soon as a second router appears.

4.6 Testing Connectivity and Troubleshooting

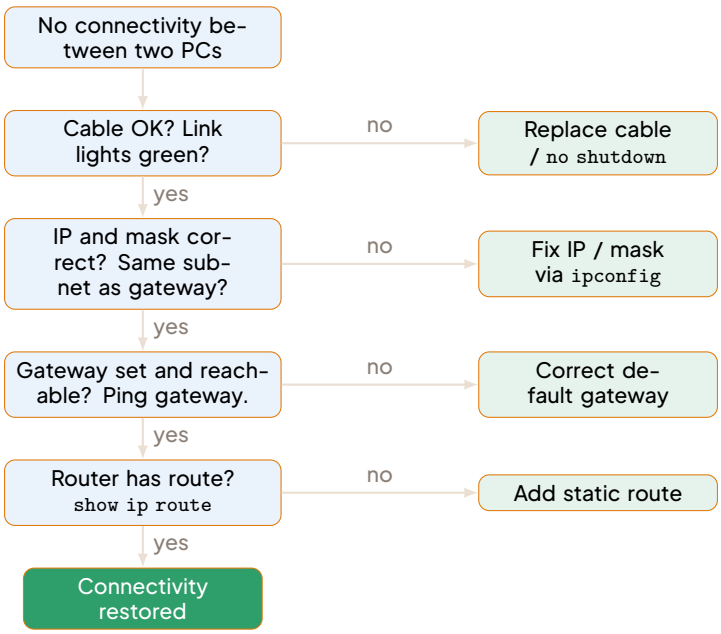
From a PC's command prompt (Desktop → Command Prompt) we use three diagnostic commands:

- `ipconfig` (Windows) / `ifconfig` or `ip a` (Linux): shows the PC's own IP, mask and gateway — **always check this first**.
- `ping <address>`: tests end-to-end connectivity (ICMP Echo Request/Reply). Ping sequence: **1. own IP, 2. default gateway, 3. remote PC**.
- `tracert <address>` (Windows) / `traceroute` (Linux): shows each router hop on the path, revealing where packets stop. In Packet Tracer, use Simulation Mode to watch the TTL decrement.

METHOD **Systematic troubleshooting order — bottom-up OSI**

1. Physical layer (Layer 1): is the cable the right type and properly seated? Are the link lights green? Is the interface `no shutdown`? (`show interfaces status`)
2. Data Link layer (Layer 2): correct VLAN? (not in this lab) Port security not blocking? MAC address table correct? (`show mac address-table`)

3. Network layer (Layer 3): correct IP, correct mask, correct gateway (`ipconfig`); do the PC and gateway belong to the same subnet? (`show ip route` on router)
4. Transport/Application (Layers 4–7): is an ACL or firewall blocking the traffic? (`show access-lists`, `show running-config`)



Troubleshooting decision tree: always work upwards from the physical layer to routing.

WORKED EXAMPLE 5 **Diagnosing a failed ping**

PC1 (192.168.10.10/26, gateway 192.168.10.1) cannot ping PC3 (192.168.10.74). `ipconfig` on PC3 shows IP `192.168.10.74`, mask `255.255.255.0`, gateway `192.168.10.65`. Find the fault.

SOLUTION

Solution. The mask on PC3 is wrong: `/24` instead of `/26`. With a /24 mask, PC3 believes 192.168.10.10 is on its own subnet, so it sends replies directly to its local switch (ARP for 192.168.10.10) instead of forwarding them to the gateway 192.168.10.65; the replies never reach PC1. Correct the mask to `255.255.255.192` and the ping succeeds. Lesson: a wrong mask can break communication even when every address looks plausible.

WORKED EXAMPLE 6 **One-way ping: PC1 pings PC3, but PC3 cannot ping PC1**

All IPs and masks are correct. `show ip route` on R1 shows both connected routes. What is missing?

SOLUTION

Solution. The return path. PC1's ping reaches PC3 because R1 knows the 192.168.10.64/26 network (connected). But if PC3's gateway (R1's G0/1) had no route back to 192.168.10.0/26, the reply would be dropped. In our single-router lab both interfaces are on the same router, so the return path exists automatically. In a multi-router lab, every router must have a route to every subnet. Always verify the routing table on both ends.

4.7 Applying Security in the Lab

The lab lets us apply the security goals of Section 2 concretely:

- **Device passwords**: protect the router with `enable secret <password>` (stored as MD5 hash, irreversible) and a console line password, so that only authorised staff can reconfigure it.

```
R1(config)\# enable secret class
R1(config)\# line console 0
R1(config-line)\# password cisco
R1(config-line)\# login
R1(config-line)\# logging synchronous
```

- **SSH instead of Telnet**: Telnet sends passwords in clear text (visible in Packet Tracer's simulation mode), whereas SSH encrypts them. Configure SSH:

```
R1(config)\# ip domain-name school.cm
R1(config)\# crypto key generate rsa modulus 1024
R1(config)\# line vty 0 4
R1(config-line)\# transport input ssh
R1(config-line)\# login local
R1(config)\# username admin privilege 15 secret adminpass
```

- **Basic ACL demonstration**: a standard access list can block one subnet from pinging the other, e.g. `access-list 1 deny 192.168.10.64 0.0.0.63` then `access-list 1 permit any`, applied **inbound** on G0/0. Pings from Science to Administration then fail while other traffic passes — a first taste of firewall behaviour.

```
R1(config)\# access-list 1 deny 192.168.10.64 0.0.0.63
R1(config)\# access-list 1 permit any
R1(config)\# interface g0/0
R1(config-if)\# ip access-group 1 in
```

- **Port security (switch)**: limit the number of MAC addresses per port and shut down the port on violation.

```
S1(config)\# interface range fa0/1 - 2
S1(config-if-range)\# switchport mode access
S1(config-if-range)\# switchport port-security
S1(config-if-range)\# switchport port-security maximum 1
S1(config-if-range)\# switchport port-security violation shutdown
S1(config-if-range)\# switchport port-security mac-address sticky
```

WARNING **Lab discipline**

Always save your work (`copy running-config startup-config` on the router, and save the Packet Tracer file `.pkt`), and document every change before testing it. An undocumented change that breaks the network is almost impossible to undo efficiently — exactly as in real professional practice.

4.8 Documenting the Lab: the Technical Report

Professional network work is judged as much by its documentation as by its configuration. Your lab report should contain: a network diagram, an **addressing table** (device, interface, IP, mask, gateway), and a **test results table** (test, expected result, observed result, pass/fail).

METHOD **Standard lab report structure**

1. Objective: what the lab sets out to achieve (one paragraph).
2. Topology: the labelled network diagram (drawn neatly or exported from Packet Tracer).
3. Addressing table: the planned IP plan (see table below).
4. Configuration steps: the essential commands used (not every keystroke, but the logical steps).
5. Tests: ping/tracert results, with screenshots or transcripts; include at least one failed test and its resolution.
6. Conclusion: difficulties met, faults diagnosed, lessons learned, and how this maps to the SDLC.

Device	Interface	IP Address	Subnet Mask	Default Gateway
R1	G0/0	192.168.10.1	255.255.255.192	—
R1	G0/1	192.168.10.65	255.255.255.192	—
PC1	NIC	192.168.10.10	255.255.255.192	192.168.10.1
PC2	NIC	192.168.10.11	255.255.255.192	192.168.10.1
PC3	NIC	192.168.10.74	255.255.255.192	192.168.10.65
PC4	NIC	192.168.10.75	255.255.255.192	192.168.10.65

4.9 Synthesis: From the Lab Back to the Whole Chapter

This single lab touches every theme of the chapter. Cabling and link lights belong to the **physical layer** of the OSI model; switches and MAC forwarding to the **data link layer**; IP addressing, subnetting and routing to the **network layer**; ping and traceroute ride on transport (ICMP) and application protocols. The passwords, ACLs, SSH and port security embody the security goals of **confidentiality, integrity and availability**. Finally, the way we ran the lab — plan the addressing, implement, test, document — is nothing other than the **SDLC** applied to infrastructure: analysis (requirements of the four departments), design (subnet plan and topology), implementation (configuration), testing (ping, troubleshooting), and maintenance (documentation for future changes). Mastering this virtuous circle is exactly what the GCE Advanced Level rewards.

EXERCISE 4 **Full subnetting and design**

The network `10.20.0.0/24` must serve 6 departments of a college in Douala. (a) How many bits must be borrowed, and what is the new mask? (b) List the network and broadcast addresses of the first three subnets. (c) Choose gateway addresses and propose an addressing table for two PCs in the first subnet. (d) State one security measure you would apply to the router and justify it.

SOLUTION **Exercise 1**

(a) $2^n \geq 6$ gives $n = 3$ bits; new prefix /27; mask `255.255.255.224`. (b) Subnet 1: network `10.20.0.0`, broadcast `10.20.0.31`; subnet 2: `10.20.0.32` / `10.20.0.63`; subnet 3: `10.20.0.64` / `10.20.0.95`. (c) Gateway `10.20.0.1`; PCs `10.20.0.10` and `10.20.0.11`, mask `255.255.255.224`, gateway `10.20.0.1`. (d) For example an `enable secret` password plus SSH-only remote access: it protects confidentiality of management traffic and prevents unauthorised reconfiguration.

EXERCISE 5 **VLSM design**

A network `172.16.0.0/24` must accommodate: LAN A (50 hosts), LAN B (20 hosts), LAN C (10 hosts), and two router-to-router serial links (2 hosts each). Design a VLSM addressing scheme. List each subnet with network address, mask, usable range, and broadcast.

SOLUTION **Exercise 2**

Order by size: 50, 20, 10, 2, 2.

- LAN A (50) → /26 (64 block): `172.16.0.0/26`, usable `.1`–`.62`, broadcast `.63`

- LAN B (20) → /27 (32 block): `172.16.0.64/27`, usable `.65--.94`, broadcast `.95`
- LAN C (10) → /28 (16 block): `172.16.0.96/28`, usable `.97--.110`, broadcast `.111`
- Serial 1 (2) → /30 (4 block): `172.16.0.112/30`, usable `.113--.114`, broadcast `.115`
- Serial 2 (2) → /30 (4 block): `172.16.0.116/30`, usable `.117--.118`, broadcast `.119`

All subnets are disjoint and fit within the original /24.

EXERCISE 6 **Troubleshooting scenario**

In the lab topology, PC1 (192.168.10.10/26) can ping its gateway (192.168.10.1) but cannot ping PC3 (192.168.10.74/26). `show ip route` on R1 shows:

```
C    192.168.10.0/26 is directly connected, GigabitEthernet0/0
C    192.168.10.64/26 is directly connected, GigabitEthernet0/1
```

`show interfaces g0/1` shows `line protocol is down`. What is the most likely cause and how do you fix it?

SOLUTION **Exercise 3**

The routing table is correct (both connected routes present). `line protocol is down` on G0/1 means the Data Link layer is not operational. Most likely cause: the cable between R1 G0/1 and Switch 2 is unplugged, wrong cable type (crossover instead of straight-through), or the switch port is shut down. Fix: verify physical connection, ensure straight-through cable, and on Switch 2 run `no shutdown` on the port connected to R1.

5 INTEGRATION ACTIVITY

5.1 Aims of the activity

This integration activity closes the chapter by asking you to combine, in a single coherent project, everything you have studied in Lessons 101 to 103. By the end of the activity, you should be able to:

- **subnet** a given IPv4 address block into several subnets and produce a complete, correct addressing table;
- design a small **LAN** topology with routers and switches, configure gateways and static routes, and test connectivity in a simulator (Cisco Packet Tracer);
- identify realistic threats to a school network and propose security measures explicitly mapped to the **CIA triad** (Confidentiality, Integrity, Availability);
- select and justify an **SDLC model** for a small management application, and write a test plan containing **normal**, **boundary** and **erroneous** test data;
- communicate technical decisions in a short written report and an oral defence, as required in the GCE Advanced Level problem-solving approach.

KEY POINTS **Skills mobilised**

Addressing and subnetting (VLSM-free, fixed-length subnetting), routing and gateways, network security and the CIA triad, software life-cycle models, software testing, technical documentation. No single skill is enough: the project only succeeds when all of them fit together.

5.2 Situation

EXPERIMENT **Networking the Lycée Bilingue de Bafoussam**

The Lycée Bilingue de Bafoussam (West Region) is a growing public secondary school with about 1800 students. The school occupies three buildings connected by fibre-optic links installed during a recent renovation:

- Building A – Administration: the principal's office, the bursar's office and the secretariat; 20 computers are needed now, with growth expected.
- Building B – Classrooms: 12 multimedia classrooms, each with one teacher's computer and a projector controller; 30 devices in total.
- Building C – Computer Laboratory: two practical rooms used for Computer Science and for the GCE practical sessions; 60 computers.

The Internet Service Provider has allocated the school the private address block 172.16.0.0/16, to be used internally behind a router that connects the campus to the Internet. The administration imposes the following requirements:

1. Each building must have its own subnet, so that a broadcast storm in the laboratory cannot disturb the bursar's office. Subnets must be large enough for current needs plus reasonable growth.
2. The three subnets are interconnected by a central router (one interface per subnet) placed in Building A; the router also provides the link to the Internet.
3. Administrative traffic (student records, fees, examination marks) must be protected: the school has already suffered one incident in which a student changed his own term average in a shared spreadsheet.
4. The school wants to replace the paper register with a student-records application (enrolment, marks entry, report-card printing). A local software company will develop it, but the school must specify the development approach and the acceptance tests.

Your team of four students has been hired as junior network consultants. You must design the network, simulate it, secure it, plan the software, and defend your choices.

5.3 Tasks

Work in teams. Answer the tasks in order; each one depends on the previous.

Task 1 – Subnetting and addressing table.

1. How many bits must be borrowed from the host part of 172.16.0.0/16 to obtain at least 3 subnets? Give the new subnet mask in dotted-decimal and prefix notation.
2. Using the first three subnets, complete an addressing table with, for each building: subnet address, subnet mask, first usable host address, last usable host address, broadcast address, and the number of usable hosts.
3. Check that each subnet can hold the number of devices of its building (20, 30, 60 hosts). If one cannot, revise your borrowing.
4. Decide a convention: the **first usable address** of each subnet will be the router interface (default gateway). Write down the gateway address of each building.

Task 2 – Simulation in Packet Tracer.

1. Draw the topology: one router, three switches (one per building), and at least two PCs per subnet.
2. Configure each PC with an IP address, mask and gateway taken from your Task 1 table; configure the three router interfaces.
3. Add a fourth network representing the ISP link (e.g. 200.100.50.0/30) and configure the

necessary **static route(s)** so that PCs can reach a simulated Web server on the ISP side.

4. Test with `ping` and `tracert`: PC in Building C → PC in Building A; PC in Building B → Web server. Record screenshots and explain any failure you met and how you fixed it.

Task 3 – Security plan.

1. List three concrete threats to this school network (think of the marks-changing incident).
2. Propose **three security measures**, and for each one state clearly which pillar(s) of the CIA triad it protects and how.
3. Suggest where each measure is applied (router, server, user workstations, or people/procedures).

Task 4 – Software engineering plan.

1. Choose an SDLC model (waterfall, V-model, incremental, or prototyping) for the student-records application and justify your choice with at least two arguments linked to this precise context.
2. Write the first three phases of your chosen model as they would apply here (one or two sentences each).
3. Design a test plan for the marks entry module (a mark is an integer from 0 to 20): give at least one **normal** test case, two **boundary** test cases and two **erroneous** test cases, with the expected result of each.

Task 5 – Report and defence. Produce a two-page technical report (diagram, addressing table, security table, SDLC justification, test plan) and defend it in a 5-minute presentation. Every team member must speak.

5.4 Model answer

Task 1 – Subnetting.

METHOD **Fixed-length subnetting (step by step)**

1. Write the given network in binary: 172.16.0.0/16 = `10101100.00010000.00000000.00000000` with mask `11111111.11111111.00000000.00000000`.
2. Determine bits to borrow: we need ≥ 3 subnets. $2^1 = 2$ (insufficient), $2^2 = 4$ (sufficient). Borrow 2 bits.
3. New prefix length: 16 + 2 = 18 bits → /18.
4. New mask: first 18 bits set to 1 → `11111111.11111111.11000000.00000000` = 255.255.192.0.
5. Block size (increment) in the interesting octet (3rd octet): $2^{8-2} = 2^6 = 64$.
6. Subnet addresses: 172.16.0.0, 172.16.64.0, 172.16.128.0, 172.16.192.0.

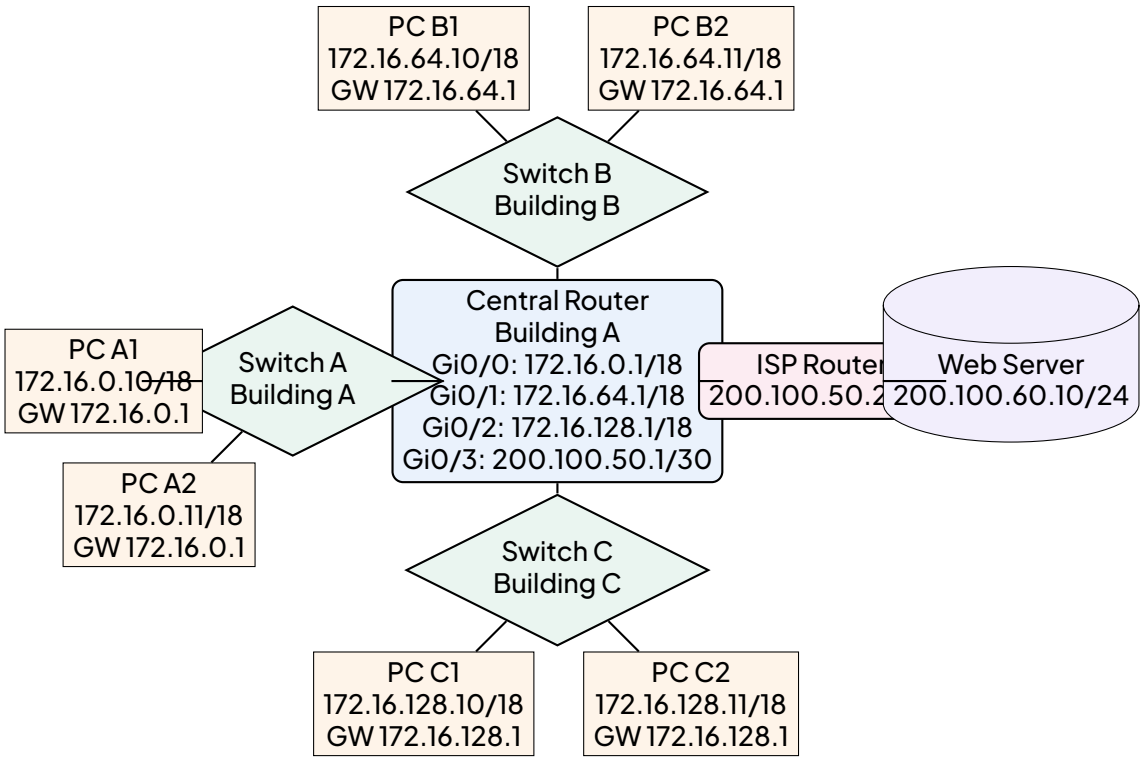
7. For each subnet: first usable = subnet +1; last usable = next subnet −2; broadcast = next subnet −1; usable hosts = $2^{14} - 2 = 16\,382$.

The block 172.16.0.0/16 has 16 host bits. We need at least 3 subnets, so we borrow **2 bits**: $2^2 = 4$ subnets ≥ 3. The new prefix is /18, i.e. mask **255.255.192.0**. Each subnet then has 16 − 2 = 14 host bits, giving $2^{14} - 2 = 16\,382$ usable hosts – far more than needed, which is acceptable here (a stricter VLSM design is possible but not required by the syllabus).

Item	Building A (Admin)	Building B (Class-rooms)	Building C (Lab)
Subnet address	172.16.0.0	172.16.64.0	172.16.128.0
Mask / prefix	255.255.192.0 /18	255.255.192.0 /18	255.255.192.0 /18
First usable	172.16.0.1	172.16.64.1	172.16.128.1
Last usable	172.16.63.254	172.16.127.254	172.16.191.254
Broadcast	172.16.63.255	172.16.127.255	172.16.191.255
Usable hosts	16 382	16 382	16 382
Gateway (router)	172.16.0.1	172.16.64.1	172.16.128.1

Capacity check: 16 382 ≥ 60 (the largest demand), so all three buildings fit comfortably with room for growth. The fourth subnet, 172.16.192.0/18, is kept in reserve for future expansion (e.g., a new building or Wi-Fi network).

Task 2 – Simulation.



Topology for Lycée Bilingue de Bafoussam network simulation

Example configuration for a lab PC in Building C: IP 172.16.128.10, mask 255.255.192.0, gateway 172.16.128.1. The router's three LAN interfaces use the gateway addresses from Task 1. The ISP link uses 200.100.50.0/30: school router Gi0/3 = 200.100.50.1, ISP router = 200.100.50.2; the Web server sits behind the ISP router at 200.100.60.10/24.

On the school router we add the static route for the Internet side:

- `ip route 200.100.60.0 255.255.255.0 200.100.50.2`

On the ISP router we add the return route:

- `ip route 172.16.0.0 255.255.0.0 200.100.50.1`

(Using 255.255.0.0 summarises the whole school block; alternatively, three /18 routes work equally well.)

Tests:

- `ping 172.16.0.10` from PC C1 (172.16.128.10) succeeds – inter-subnet routing through the central router.
- `tracert 200.100.60.10` from PC B1 shows two hops: 172.16.64.1 (school router) then 200.100.50.2 (ISP router).

WARNING **Frequent errors in the lab**

Wrong mask on one PC only; gateway pointing to a host instead of the router interface; static route configured in one direction only (the reply never comes back); using the subnet address or the broadcast address for a host; forgetting to enable the router interfaces (`no shutdown`).

Task 3 – Security plan (mapped to the CIA triad).

DEFINITION **CIA Triad**

The three core principles of information security:

- Confidentiality: Only authorised users can view data.
- Integrity: Data is accurate and cannot be altered by unauthorised parties.
- Availability: Systems and data are accessible when needed.

Threat	Measure	CIA pillar(s)	Where applied
A student modifies marks (the past incident)	Individual user accounts with strong passwords and role-based access control (RBAC) on the records server; only teachers may write marks	**Confidentiality** and **Integrity**	Server (application & database)
Data loss (fire, disk failure, theft of the server)	Automatic daily backups of the records database to an external disk kept in a different building (Building B or C)	**Availability**	Procedure + backup server
Virus or intruder entering from the Internet or an infected USB key	Firewall on the Internet router (default deny, only allow HTTP/HTTPS/DNS to the Web server) plus updated antivirus on every workstation	**Integrity** and **Availability**	Router (firewall) + Workstations (antivirus)

The access-rights measure is applied on the server, the firewall on the router, the antivirus on workstations, and backups are a procedure – reminding us that security is technical and organisational.

Task 4 – Software engineering plan.

Chosen model: the **incremental model**. Justification: (1) the school needs the enrolment module quickly (start of the school year in September) and can receive marks entry and report-card printing in later increments; (2) the bursar and teachers are not computer specialists, so delivering small working versions lets them give feedback early and correct misunderstandings before they become expensive. A pure waterfall model would deliver everything at once, too late and with no intermediate feedback; the V-model is too rigid for evolving requirements; prototyping alone lacks the structured increments needed for a production system.

First phases applied:

1. **Analysis:** Interview the principal, bursar and teachers; list functional requirements (register a student, enter a mark 0–20, print a report card) and non-functional requirements (response time < 2 s, backup daily, role-based access).
2. **Design:** Database schema (tables Student, Subject, Mark, User), ER diagram, screen mock-ups for each module, validated by the users before coding starts.
3. **Implementation of Increment 1:** Develop the enrolment module (CRUD for students), unit-test it, install on the school server, train the secretariat, gather feedback.

Test plan for marks entry (valid mark: integer, 0 to 20):

Type	Input	Expected result	Why
Normal	12	Mark accepted and saved	Ordinary value in the middle of the range
Boundary	0	Accepted	Smallest valid mark (lower boundary)
Boundary	20	Accepted	Largest valid mark (upper boundary)
Erroneous	-1	Rejected with error message “Mark must be between 0 and 20”	Just below the range
Erroneous	21	Rejected with error message “Mark must be between 0 and 20”	Just above the range
Erroneous	“abc”	Rejected with error message “Invalid input: numeric value required”	Non-numeric input (data type error)

Task 5 – Report and defence.

The two-page report contains:

- Page 1: the topology diagram (as above) and the addressing table (Task 1).
- Page 2: the security table (Task 3), the SDLC justification and the test plan (Task 4).

In the 5-minute defence, a strong team explains why /18 was chosen (2 bits borrowed → 4 subnets, each with 16 382 hosts, meeting growth requirements), demonstrates one successful cross-

subnet `ping` (e.g., PC C1 to PC A1), links each security measure to a CIA pillar explicitly, and defends the incremental model against the examiner's question "why not waterfall?" by citing the need for early enrolment delivery and user feedback.

KEY POINTS **What the examiner looks for**

A correct and complete addressing table; gateways consistent with the table; static routes in both directions; security measures explicitly tied to Confidentiality, Integrity and Availability; a justified SDLC choice with context-specific arguments; and a test plan that really distinguishes normal, boundary and erroneous data. Precision and justification matter more than volume.

6 METHODS AND WORKED EXAMPLES

7 CSC19: NETWORKING, SECURITY, SOFTWARE ENGINEERING & SYNTHESIS

8 DATA COMMUNICATION & NETWORK FUNDAMENTALS

8.1 Skill 1: Converting between binary and decimal (IP octets)

METHOD **Binary–decimal conversion of an octet**

An IPv4 address is made of four **octets** (8 bits each), each ranging from 0 to 255.

1. Decimal → binary: write the 8 place values 128, 64, 32, 16, 8, 4, 2, 1. Starting from the left, put a 1 under each value you can subtract without going negative, otherwise 0.
2. Binary → decimal: add the place values of the positions containing a 1.
3. Check: the result must lie in [0, 255]; if you obtain more than 8 bits or a value above 255, you made an error.
4. Reflex: memorise the powers of two up to $2^8 = 256$; they reappear in every subnetting question.

WORKED EXAMPLE 7 **Converting an IP address**

A host in a cybercafé in Bamenda has the IPv4 address 192.168.10.130.

1. Convert the address to binary (dotted octet form).
2. Convert the binary octet 10101100 back to decimal.

SOLUTION

1. Decimal to binary, octet by octet.
Octet 192: 192 − 128 = 64 (bit 1), 64 − 64 = 0 (bit 1), remaining bits 0.

$$192 = 11000000_2$$

Octet 168: 168 − 128 = 40 (bit 1), 40 < 64 (bit 0), 40 − 32 = 8 (bit 1), 8 < 16 (bit 0), 8 − 8 = 0

(bit 1), rest 0.

$$168 = 10101000_2$$

Octet 10: $10 = 8 + 2$, so $10 = 00001010_2$.
Octet 130: $130 - 128 = 2$ (bit 1), then 2 (bit 1 at position 2).

$$130 = 10000010_2$$

Full address: `11000000.10101000.00001010.10000010`.
2. Binary to decimal.

$$10101100_2 = 128 + 32 + 8 + 4 = 172$$

Check: $128 + 0 + 32 + 0 + 8 + 4 + 0 + 0 = 172$. Correct, and $0 \le 172 \le 255$.

8.2 Skill 2: Computing transmission time and throughput

METHOD **Data communication calculations**

1. Identify the **file size** S (convert to bits: 1 byte = 8 bits; 1 KB = 1024 bytes) and the **bandwidth** B in bits per second.
2. Apply the fundamental formula:

$$\text{transmission time } t = \frac{S}{B} \quad \Longleftrightarrow \quad B = \frac{S}{t}$$

3. If **latency** (propagation delay) t_p is given, total time = $t_p + t$.
4. Convert units coherently before substituting (Mbps means megabits per second, not megabytes).

WORKED EXAMPLE 8 **Downloading an examination timetable**

A student in Yaoundé downloads a PDF file of 2.5 MB over a connection of bandwidth 4 Mbps. The network latency is 120 ms.

1. Convert the file size to bits.
2. Calculate the transmission time.
3. Calculate the total download time.

SOLUTION

1. File size in bits.

$$S = 2.5 \times 1024 \times 1024 \times 8 = 20\,971\,520 \text{ bits}$$

2. Transmission time. Bandwidth $B = 4$ Mbps $= 4 \times 10^6$ bits/s.

$$t = \frac{S}{B} = \frac{20\,971\,520}{4 \times 10^6} \approx 5.24 \text{ s}$$

3. Total time. Latency $t_p = 120$ ms $= 0.12$ s.

$$t_{\text{total}} = t_p + t = 0.12 + 5.24 = 5.36 \text{ s}$$

Conclusion: the student waits about 5.4 seconds. Note that the latency is negligible here, but it dominates for very small files.

DID YOU KNOW? **Bandwidth vs. throughput in Cameroon**

In practice, the throughput (useful data rate) is lower than the nominal bandwidth because of protocol headers (Ethernet, IP, TCP), retransmissions, and shared medium contention. A 4 Mbps subscription in Douala or Bafoussam typically yields 3.0–3.5 Mbps of actual throughput for TCP downloads.

8.3 Skill 3: Analysing an IPv4 address (class, network, broadcast)

METHOD **Reading an IPv4 address with its mask**

1. Write the address and the **subnet mask** in binary.
2. The mask's 1-bits define the **network prefix**; the 0-bits define the host part. A mask of n ones is written $/n$ (CIDR notation).
3. Network address: perform a bitwise AND between address and mask (equivalently: keep the prefix, set host bits to 0).
4. Broadcast address: keep the prefix, set all host bits to 1.
5. Number of hosts: with h host bits, there are $2^h - 2$ usable addresses (subtract network and broadcast addresses).
6. Recall the classes: A (1–126, /8), B (128–191, /16), C (192–223, /24); 127.*x*.*x*.*x* is loopback.

WORKED EXAMPLE 9 **Analysing a school network address**

The computer laboratory of a Government High School uses the address 192.168.15.77 with mask 255.255.255.0.

1. Give the class of this address and its CIDR prefix.
2. Determine the network address and the broadcast address.
3. How many machines can be connected?

SOLUTION

1. Class and prefix. The first octet is 192, which lies in [192, 223]: this is a **class C** address. The mask 255.255.255.0 contains 24 ones, so the prefix is /24.
2. Network and broadcast. The first three octets form the network part; the last octet is the host part.
   - Network address: host bits set to 0 ⇒ 192.168.15.0.
   - Broadcast address: host bits set to 1 ⇒ 192.168.15.255.
3. Number of hosts. There are $h = 32 - 24 = 8$ host bits:

$$2^8 - 2 = 256 - 2 = 254 \text{ usable addresses}$$

So up to 254 machines (computers, printer, server) can be connected in this laboratory network.

9 IP ADDRESSING & NETWORK SECURITY

9.1 Skill 4: Subnetting a network into equal parts

METHOD **Subnetting with borrowed bits**

1. To create k subnets, borrow b bits from the host part such that $2^b \geq k$ (take the smallest such b).
2. New prefix length = old prefix + b. Write the new mask.
3. The **block size** (step between subnets) is $2^{\text{remaining host bits in the octet being cut}}$, i.e. 256 − mask octet value for a /24 base.
4. List the subnets by adding the block size repeatedly in the relevant octet.

5. For each subnet give: network address, first usable, last usable, broadcast, and number of hosts $2^h - 2$.

WORKED EXAMPLE 10 **Subnetting a school network**

A school receives the network 192.168.1.0/24 and must create 4 subnets: Administration, Staff, Students, and Library.

1. How many bits must be borrowed? Give the new mask in decimal and CIDR form.
2. List the four subnets with their network and broadcast addresses.
3. How many hosts per subnet?

SOLUTION

1. Borrowed bits. We need $2^b \geq 4$, so $b = 2$. New prefix: 24 + 2 = 26, i.e. /26. The mask has 26 ones: the last octet is $11000000_2 = 192$.

$$\text{New mask} = 255.255.255.192 \quad (/26)$$

2. Subnets. Block size = 256 − 192 = 64 in the last octet:

Subnet	Network	Usable range	Broadcast
Administration	192.168.1.0	192.168.1.1 – 192.168.1.62	192.168.1.63
Staff	192.168.1.64	192.168.1.65 – 192.168.1.126	192.168.1.127
Students	192.168.1.128	192.168.1.129 – 192.168.1.190	192.168.1.191
Library	192.168.1.192	192.168.1.193 – 192.168.1.254	192.168.1.255

3. Hosts per subnet. Remaining host bits: $h = 32 - 26 = 6$.

$$2^6 - 2 = 62 \text{ hosts per subnet}$$

Each department can therefore connect up to 62 devices, well isolated from the others.

WARNING **Classic subnetting traps**

- Forgetting to subtract 2 for the network and broadcast addresses.

- Confusing the number of subnets (2^b) with the number of hosts.
- Writing a mask whose ones are not contiguous (e.g. 255.255.0.255 is invalid).

9.2 Skill 5: Applying a Caesar cipher and discussing security mechanisms

METHOD **Encrypting and decrypting with a Caesar cipher**

1. Number the letters $A = 0, B = 1, \ldots, Z = 25$.
2. Encryption with key k: $C = (P + k) \bmod 26$.
3. Decryption: $P = (C - k) \bmod 26$ (add 26 if the result is negative).
4. Security reflex: a Caesar cipher has only 25 useful keys, so it is broken instantly by **brute force**; real systems use long keys (symmetric, e.g. AES) or public/private key pairs (asymmetric, e.g. RSA), plus firewalls, authentication and antivirus software.

WORKED EXAMPLE 11 **Encrypting a password hint**

Using a Caesar cipher with key $k = 3$:

1. Encrypt the message `EXAM`.
2. Decrypt the ciphertext `FDU`.
3. Explain why this cipher is not suitable for protecting a bank account, and name two stronger security measures.

SOLUTION

1. Encryption of EXAM ($C = (P + 3) \bmod 26$):
   - $E = 4 \rightarrow 7 = H$
   - $X = 23 \rightarrow 26 \bmod 26 = 0 = A$
   - $A = 0 \rightarrow 3 = D$
   - $M = 12 \rightarrow 15 = P$

   Ciphertext: `HADP`.
2. Decryption of FDU ($P = (C - 3) \bmod 26$):
   - $F = 5 \rightarrow 2 = C$
   - $D = 3 \rightarrow 0 = A$

- $U = 20 \rightarrow 17 = R$

Plaintext: `CAR`.
3. Weakness and stronger measures. There are only 25 possible keys, so an attacker can try them all in seconds (brute-force attack); letter frequency analysis also breaks it easily. Stronger measures include: a modern encryption algorithm with a long key (AES), public-key cryptography (RSA) for secure exchange, a firewall to filter traffic, and strong passwords with two-factor authentication.

DID YOU KNOW? **From Caesar to TLS in Cameroon**

Modern Cameroonian banking apps (e.g., mobile money platforms) use TLS 1.2/1.3, which combines asymmetric cryptography (RSA or ECDHE) for key exchange and symmetric encryption (AES-GCM) for data confidentiality. A Caesar cipher would be broken before the SMS OTP arrives.

10 SOFTWARE ENGINEERING, SDLC & THEORETICAL COMPUTER SCIENCE

10.1 Skill 6: Choosing an SDLC model and justifying the choice

METHOD **Selecting a software development life cycle model**

1. List the **project characteristics**: are requirements fixed and well understood? Is the project large or small? Does the client need to see early versions? Is risk high?
2. Match with a model:
   - Waterfall: requirements clear and stable, small well-defined project; phases follow strictly (analysis → design → coding → testing → maintenance).
   - Iterative / incremental: product delivered in versions, feedback after each release.
   - Prototyping: requirements vague; a mock-up is shown to users to refine needs.
   - Spiral: large, expensive, high-risk projects; each loop includes risk analysis.
   - Agile (e.g. Scrum): changing requirements, close customer collaboration, short sprints.
3. Justify the choice with at least two arguments drawn from the scenario (GCE examin-

ers reward justification, not just the name).

WORKED EXAMPLE 12 **Choosing a model for a school fees system**

A software company in Douala must develop a school fees management system for a college. The bursar knows exactly what he wants: registration of payments, receipts, and termly balance reports. The requirements are stable and precisely documented. The project is of moderate size.

1. Name the phases of the waterfall model in order.
2. Which SDLC model do you recommend? Justify with two arguments.
3. State one disadvantage of the chosen model.

SOLUTION

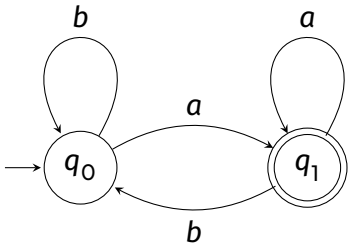
1. Waterfall phases: requirements analysis → system design → implementation (coding) → testing → deployment → maintenance. Each phase ends before the next begins, like water falling down steps.
2. Recommended model: the waterfall model.
   - Argument 1: the requirements are clear, complete and stable (payments, receipts, reports are precisely documented), so there is no need for prototypes or frequent client feedback loops.
   - Argument 2: the project is of moderate size and low risk; the heavy risk analysis of the spiral model or the overhead of agile sprints would bring no benefit.
3. Disadvantage: the waterfall model is rigid — if a requirement changes late (for example the college later demands mobile-money payment integration), returning to an earlier phase is costly and slow. Testing also happens only near the end, so defects are discovered late.

10.2 Skill 8: Synthesis — reasoning with finite automata and logic in theoretical computer science

METHOD **Analysing a finite automaton**

1. Identify: the set of **states**, the **alphabet**, the **transition function**, the **initial state** (arrow pointing in) and the **accepting states** (double circles).
2. To test a word: start at the initial state and follow the transition labelled by each symbol, one after another.

3. The word is **accepted** if and only if you finish in an accepting state after consuming all symbols.
4. To describe the language: look for the pattern common to all accepted words (e.g. "ends with 1", "contains an even number of a's").



Finite automaton accepting words with an odd number of a's over $\{a, b\}$.

WORKED EXAMPLE 13 **Testing words on a finite automaton**

Consider the automaton over the alphabet $\{a, b\}$ with states $\{q_0, q_1\}$, initial state q_0, accepting state q_1, and transitions:

$$\delta(q_0, a) = q_1, \quad \delta(q_0, b) = q_0, \quad \delta(q_1, a) = q_1, \quad \delta(q_1, b) = q_0.$$

1. Trace the processing of the words bba and $abab$.
2. Which words does this automaton accept? Justify.

SOLUTION

1. Traces.
Word bba:

$$q_0 \xrightarrow{b} q_0 \xrightarrow{b} q_0 \xrightarrow{a} q_1$$

We end in q_1, the accepting state: bba is accepted.
Word $abab$:

$$q_0 \xrightarrow{a} q_1 \xrightarrow{b} q_0 \xrightarrow{a} q_1 \xrightarrow{b} q_0$$

We end in q_0, not accepting: $abab$ is rejected.
2. Language recognised. Observe the transitions: reading a always flips the state ($q_0 \leftrightarrow q_1$), while reading b leaves the state unchanged. Starting from q_0, the automaton is in q_1 exactly when an odd number of a's has been read. Therefore:

$$L = \{\, w \in \{a, b\}^* \mid w \text{ contains an odd number of } a \,\}$$

Check with our traces: bba has one a (odd, accepted); $abab$ has two a's (even, rejected). This parity reasoning is a classic GCE-style automaton question: always ask "what does

each state remember?" — here, q_0 remembers "even so far" and q_1 remembers "odd so far".

11 HANDS-ON NETWORKING LAB: SUBNETTING & LAN SIMULATION

11.1 Skill 7: Simulating a LAN and diagnosing connectivity (practical lab)

METHOD **Building and testing a LAN in a simulator (e.g. Cisco Packet Tracer)**

1. Place devices: PCs, one switch, optionally a router; connect PCs to the switch with straight-through copper cables (cross-over between two switches or PC-to-PC).
2. Address the hosts: give each PC an IP address in the same subnet (same network address), with the correct mask; check that no two PCs share an address.
3. Test connectivity: use the `ping` command from one PC to another. A reply proves physical cabling, addressing and mask are coherent.
4. Diagnose failures systematically: cable type? interface up? same subnet? mask identical? IP duplicated?
5. Extend: to link two subnets, add a router, one interface per subnet, and set each PC's **default gateway** to its router interface.

WORKED EXAMPLE 14 **Diagnosing a failed ping**

In a Packet Tracer lab, PC1 has address 192.168.10.10/26 and PC2 has 192.168.10.70/26; both are connected to the same switch. A ping from PC1 to PC2 fails.

1. Determine the subnet of each PC.
2. Explain why the ping fails.
3. Correct the configuration so that communication works.

SOLUTION

1. Subnets. With /26, the block size is 256 - 192 = 64 in the last octet: subnets start at 0, 64, 128, 192.

- PC1: 192.168.10.10 lies in $[0, 63]$ ⇒ subnet 192.168.10.0/26 (broadcast 192.168.10.63).
- PC2: 192.168.10.70 lies in $[64, 127]$ ⇒ subnet 192.168.10.64/26 (broadcast 192.168.10.127).

2. Why the ping fails. Although the PCs are plugged into the same switch, they belong to different logical subnets. When PC1 compares its subnet (192.168.10.0/26) with the destination's, it concludes the destination is remote and tries to send the packet to a **default gateway** — but no router/gateway is configured, so the packet is dropped and the ping fails.
3. Correction (two possible solutions).

- Simplest: put both PCs in the same subnet, e.g. change PC2 to 192.168.10.20/26 (or change both masks to /24). The ping then succeeds directly through the switch.
- With routing: keep the addresses, add a router with interface 192.168.10.1/26 on the first subnet and 192.168.10.65/26 on the second, and set these as the PCs' default gateways. The router then forwards packets between the subnets.

This exercise shows the golden rule of the lab: same switch is not enough — hosts must share the same subnet or have a router between them.

WARNING **Frequent lab mistakes**

- Using a straight-through cable between two switches (use cross-over, unless auto-MDIX is simulated).
- Forgetting the default gateway when a router is present: the ping to the other subnet fails even though all cables are correct.
- Assigning the network address or the broadcast address to a PC — these are reserved.

KEY POINTS **Chapter reflexes to master**

- Powers of two up to $2^{10} = 1024$ by heart; octet conversions without hesitation.
- $t = S/B$ with coherent units (bits, not bytes).
- Subnetting: borrow b bits for 2^b subnets; block size = 256 − mask octet; hosts = $2^h - 2$.
- Security: know the difference between symmetric and asymmetric encryption, and the role of firewalls and authentication.
- SDLC: always justify the chosen model from the scenario.

- Lab: same subnet or a router with gateways — nothing else makes a ping succeed.

12 EXERCISES

12.1 I check my knowledge

EXERCISE 7 **MCQ: Networks and software engineering**

For each question, choose the single correct answer and justify your choice in one sentence.

1. In the OSI model, the layer responsible for routing packets between different networks is:
   (a) Data Link layer
   (b) Network layer
   (c) Transport layer
   (d) Application layer
2. The subnet mask corresponding to the prefix /27 is:
   (a) 255.255.255.224
   (b) 255.255.255.240
   (c) 255.255.255.192
   (d) 255.255.255.248
3. In the CIA triad, ensuring that data is not altered by unauthorised persons corresponds to:
   (a) Confidentiality
   (b) Integrity
   (c) Availability
   (d) Authentication
4. The SDLC phase in which system requirements are collected from users is:
   (a) Implementation
   (b) Testing
   (c) Analysis
   (d) Maintenance

EXERCISE 8 True or false with justification

State whether each statement is true or false, and justify your answer in one or two sentences.

1. A hub forwards frames only to the port of the destination machine.
2. The IP address 192.168.10.255 can be assigned to a host in the network 192.168.10.0/24.
3. The waterfall model allows easy return to a previous phase at any time without cost.
4. An algorithm of complexity $O(\log n)$ is generally faster than an algorithm of complexity $O(n)$ for large inputs.
5. Phishing is an attack that mainly threatens the availability of a system.

EXERCISE 9 Definitions

Define each of the following terms in one or two clear sentences, giving one example where appropriate:

1. Protocol
2. Firewall
3. Software maintenance
4. White-box testing
5. Finite automaton

EXERCISE 10 Direct calculations

1. How many usable host addresses are available in the network 10.0.0.0/28? Show your working.
2. A file of 8 MB is transmitted over a link of 4 Mbit/s. Ignoring overheads, calculate the transmission time in seconds.
3. Convert the subnet mask 255.255.255.240 into CIDR (slash) notation.
4. An algorithm executes $3n^2 + 5n + 2$ elementary operations for an input of size n. Give its complexity in Big-O notation, justifying your answer.

12.2 I practise

EXERCISE 11 OSI model in action

A student in Buea sends an email to a friend in Bamenda.

1. Name the OSI layer responsible for routing, and the layer responsible for encryption and data presentation. Justify each answer.
2. Give one protocol operating at each of these two layers.
3. For each of the seven OSI layers, give one protocol or technology associated with it.

EXERCISE 12 Designing a test table

A competition program accepts only candidates whose age is between 17 and 25 inclusive.

1. Design a test table with at least six test cases covering normal data, boundary data and erroneous data. For each case, give the input value, the category (normal / boundary / erroneous) and the expected result.
2. Explain why testing only the value 20 is not sufficient.

EXERCISE 13 Ranking complexities

1. Rank the following complexities from the most efficient to the least efficient for a growing input size n: $O(n^2)$, $O(1)$, $O(2^n)$, $O(n)$, $O(\log n)$.
2. An $O(2^n)$ algorithm takes 1 second for $n = 20$. Estimate the time needed for $n = 30$ and for $n = 40$, assuming the time is proportional to 2^n.
3. Conclude: why does an $O(2^n)$ algorithm become impractical even on a fast computer?

EXERCISE 14 Topology for a cybercafé

A cybercafé in Douala has 20 computers, one printer and one Internet router.

1. Draw (neatly, on paper) a labelled topology for this cybercafé, choosing appropriate devices (switch, router, access point if needed) and transmission media.
2. Justify your choice of topology in terms of cost, performance and ease of troubleshooting.

3. Discuss the fault tolerance of your design: what happens if the switch fails? If one cable fails? Propose one improvement to increase fault tolerance.

12.3 I prepare for the GCE

EXERCISE 15 **Subnetting problem (GCE style) — 10 marks**

A company in Yaoundé is given the network address 192.168.5.0/24. The company needs exactly 5 subnets, each supporting at least 25 hosts.

1. Determine the number of bits to borrow from the host part, and give the new subnet mask in dotted decimal and in CIDR notation. (3 marks)
2. How many subnets does this borrowing create, and how many usable hosts does each subnet support? (2 marks)
3. List the first five subnets, giving for each: the network address, the range of usable host addresses, and the broadcast address. (4 marks)
4. State one advantage of subnetting for network security. (1 mark)

EXERCISE 16 **Security scenario: the fake SMS (GCE style) — 10 marks**

Customers of a Cameroonian bank receive SMS messages pretending to come from the bank, asking them to click a link and enter their account credentials.

1. Identify the type of attack described, and name the pillar of the CIA triad that is most directly threatened. Justify. (3 marks)
2. Propose four concrete countermeasures the bank and its customers can apply, explaining briefly how each one works. (4 marks)
3. Explain the role of a firewall and of encryption in protecting online banking transactions. (2 marks)
4. Give one reason why user awareness training is considered part of security. (1 mark)

EXERCISE 17 **SDLC essay and lab diagnosis (GCE style) — 12 marks**

Part A. A startup wants to develop a mobile-money application for the Cameroonian market.

1. Compare the waterfall model and the agile model, giving two advantages and two

limitations of each. (4 marks)

2. Recommend one model for this project and justify your recommendation with three arguments related to the context (changing user needs, regulation, time to market). (3 marks)

Part B. During a Packet Tracer lab, a ping from PC1 (192.168.1.10/24) to PC2 (192.168.1.20/24) fails, although both PCs are connected to the same switch.

1. List the four most probable causes of this failure. (4 marks)

2. For each cause, give the command or check used to verify it. (1 mark)

13 SOLUTIONS TO THE EXERCISES

SOLUTION **Exercise 1 — MCQ: Networks and software engineering**

1. Answer: (b) Network layer. Routing consists in choosing a path for packets between different networks, and this is precisely the role of layer 3 (Network layer), implemented by routers using logical (IP) addresses.
2. Answer: (a) 255.255.255.224. A /27 prefix means 27 bits set to 1; the last octet has 3 bits at 1, i.e. 128 + 64 + 32 = 224.
3. Answer: (b) Integrity. Integrity guarantees that data is not modified or corrupted by unauthorised persons (confidentiality concerns unauthorised reading, availability concerns authorised access).
4. Answer: (c) Analysis. The analysis phase of the SDLC is devoted to collecting and specifying user requirements before any design or coding takes place.

SOLUTION **Exercise 2 — True or false with justification**

1. False. A hub simply repeats (broadcasts) every incoming frame on all its ports; it is the switch that forwards a frame only to the port of the destination machine, using its MAC address table.
2. False. In 192.168.10.0/24, the address 192.168.10.255 is the broadcast address of the network (all host bits at 1); it cannot be assigned to a host.
3. False. The waterfall model is strictly sequential: going back to a previous phase is difficult and costly, which is precisely one of its main limitations.
4. True. For large n, $\log n$ grows much more slowly than n (e.g. for $n = 10^6$, $\log_2 n \approx 20$), so an $O(\log n)$ algorithm performs far fewer operations than an $O(n)$ one.
5. False. Phishing aims at stealing credentials or data, so it mainly threatens confidentiality (and integrity); availability is threatened by attacks such as denial of service (DoS).

SOLUTION **Exercise 3 — Definitions**

1. Protocol: a set of rules and formats that govern how two devices exchange data over

a network. Example: HTTP for the Web, TCP for reliable transport.

2. Firewall: a hardware or software device placed between networks that filters incoming and outgoing traffic according to security rules, blocking unauthorised connections. Example: the firewall of a school network blocking certain ports.
3. Software maintenance: all activities performed on software after its delivery to correct faults, adapt it to new environments or improve it (corrective, adaptive, perfective maintenance). Example: updating a mobile-money app after a new Android version is released.
4. White-box testing: a testing technique in which the tester knows the internal structure and source code of the program and designs test cases to cover its instructions, branches and paths. Example: testing every branch of an `if` statement.
5. Finite automaton: an abstract machine made of a finite set of states, an input alphabet and transitions between states, which accepts or rejects input strings; it is used to recognise regular languages. Example: an automaton checking that a password contains at least one digit.

SOLUTION **Exercise 4 — Direct calculations**

1. A /28 network has $32 - 28 = 4$ host bits. Number of addresses: $2^4 = 16$; subtracting the network address and the broadcast address gives $16 - 2 = 14$ usable host addresses.
2. Convert the file size to bits: $8\ \text{MB} = 8 \times 8 = 64$ Mbit. Transmission time:
$$t = \frac{\text{size}}{\text{rate}} = \frac{64\ \text{Mbit}}{4\ \text{Mbit/s}} = 16\ \text{s}.$$
3. 255.255.255.240: the last octet $240 = 128 + 64 + 32 + 16$ has 4 bits at 1. Total: $24 + 4 = 28$ bits, hence the CIDR notation /28.
4. For $3n^2 + 5n + 2$, the dominant term as n grows is $3n^2$; constants and lower-order terms are ignored in Big-O notation. Hence the complexity is $O(n^2)$ (quadratic). Formally, $3n^2 + 5n + 2 \le 10n^2$ for all $n \ge 1$, which proves the bound.

SOLUTION **Exercise 5 — OSI model in action**

1. Routing: the **Network layer** (layer 3), because it selects paths between different networks using logical (IP) addresses. Encryption and data presentation: the **Presentation layer** (layer 6), because it handles data formats, compression and encryption/decryption so that two systems understand each other.

2. Network layer: IP (Internet Protocol). Presentation layer: SSL/TLS (encryption), or ASCII/JPEG as data formats.

3. One protocol or technology per layer:

Layer	Name	Example
7	Application	HTTP, SMTP (email), FTP
6	Presentation	SSL/TLS, JPEG, ASCII
5	Session	NetBIOS, PPTP
4	Transport	TCP, UDP
3	Network	IP, ICMP (ping)
2	Data Link	Ethernet (MAC), PPP, Wi-Fi framing
1	Physical	UTP cable, fibre optics, radio waves

SOLUTION **Exercise 6 — Designing a test table**

1. Test table (accept if 17 ≤ age ≤ 25):

Case	Input	Category	Expected result
1	20	Normal	Accepted
2	17	Boundary (lower, valid)	Accepted
3	25	Boundary (upper, valid)	Accepted
4	16	Boundary (just below)	Rejected
5	26	Boundary (just above)	Rejected
6	-5	Erroneous	Rejected / error message
7	“abc”	Erroneous (non-numeric)	Rejected / error message

2. Testing only the value 20 exercises just one path of the program (the “accept” branch). It cannot detect errors in the comparisons at the limits: for example a program written with `age > 17` instead of `age >= 17` would accept 20 but wrongly reject a 17-year-old candidate. Good testing must cover normal values, boundary values (where bugs most often occur) and erroneous values.

SOLUTION **Exercise 7 — Ranking complexities**

1. From most efficient to least efficient:

$$O(1) < O(\log n) < O(n) < O(n^2) < O(2^n).$$

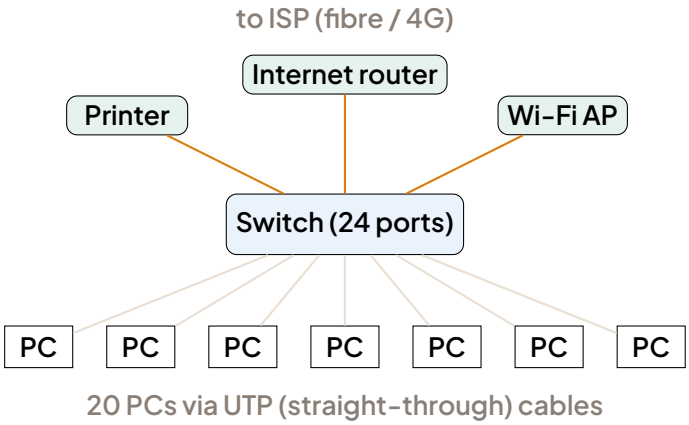
2. Since time is proportional to 2^n, multiplying n by adding 10 multiplies the time by $2^{10} = 1024$:

$$n = 30: \quad t = 1 \times 2^{30-20} = 2^{10} = 1024 \text{ s} \approx 17 \text{ minutes};$$
$$n = 40: \quad t = 1 \times 2^{40-20} = 2^{20} = 1\,048\,576 \text{ s} \approx 12 \text{ days}.$$

3. Each time n increases by just 1, the execution time doubles. The growth is exponential: even a computer 1000 times faster only allows about 10 extra units of input size ($2^{10} \approx 1000$). Hence an $O(2^n)$ algorithm quickly becomes impractical for realistic input sizes, whatever the hardware.

SOLUTION **Exercise 8 — Topology for a cybercafé**

1. A star topology centred on a switch:



Star topology for the cybercafé: 20 PCs, one printer and one Wi-Fi access point connected to a central switch, itself linked to the Internet router.

2. Justification. Cost: a star with one 24-port switch and UTP cables is cheap and uses standard equipment. Performance: the switch gives each PC a dedicated full-duplex link, so collisions are eliminated and one heavy user does not slow the others. Troubleshooting: each station has its own cable to the centre, so a faulty PC or cable is identified and isolated immediately without disturbing the rest of the network.

3. Fault tolerance. If the switch fails, the whole LAN stops (single point of failure): no PC

can communicate, although the Internet line itself may still be up. If one cable fails, only the single PC at its end is disconnected; the rest of the network keeps working — this is the great advantage of the star. Improvement: add a second switch (uplinked to the first) and split the PCs between the two switches, or keep a spare switch and spare cables on site; an uplink between two switches also removes part of the single point of failure.

SOLUTION **Exercise 9 — Subnetting problem (GCE style)**

Indicative mark scheme in brackets.

1. (3 marks: 1 for reasoning on subnets, 1 for bits borrowed, 1 for both mask forms.)
   We need at least 5 subnets. Borrowing b bits creates 2^b subnets: $2^2 = 4 < 5$ and $2^3 = 8 \geq 5$, so we borrow 3 bits from the host part.
   New prefix: $24 + 3 = 27$, i.e. /27. New mask: the last octet has 3 bits at 1: $128 + 64 + 32 = 224$, hence 255.255.255.224.

2. (2 marks: 1 each.)
   Subnets created: $2^3 = 8$ subnets. Hosts per subnet: $32 - 27 = 5$ host bits, $2^5 - 2 = 30$ usable hosts per subnet (which satisfies "at least 25").

3. (4 marks: 1 per subnet row fully correct; block size 32.) The block size is $256 - 224 = 32$:

Subnet	Network address	Usable host range	Broadcast
1	192.168.5.0	192.168.5.1 – 192.168.5.30	192.168.5.31
2	192.168.5.32	192.168.5.33 – 192.168.5.62	192.168.5.63
3	192.168.5.64	192.168.5.65 – 192.168.5.94	192.168.5.95
4	192.168.5.96	192.168.5.97 – 192.168.5.126	192.168.5.127
5	192.168.5.128	192.168.5.129 – 192.168.5.158	192.168.5.159

4. (1 mark.) Subnetting segments the network: each department is isolated in its own subnet, so a router/firewall can filter traffic between subnets and an infection or attack in one subnet is contained and does not spread directly to the others (it also reduces the broadcast domain).

WARNING **Examiner's expectations**

The candidate must justify the choice of 3 bits (and not 2), must subtract 2 for hosts, and must not include the network or broadcast addresses in the usable range. A frequent error costing marks is writing host ranges that overlap the next subnet.

SOLUTION **Exercise 10 — Security scenario: the fake SMS (GCE style)**

1. (3 marks: 1 for the attack, 1 for the pillar, 1 for justification.)
   This is a phishing attack (more precisely smishing, phishing by SMS): attackers impersonate the bank to trick customers into revealing their credentials. The CIA pillar most directly threatened is confidentiality, because the goal is to steal secret information (login, password, account data); once stolen, integrity of the accounts is also at risk.
2. (4 marks: 1 per countermeasure with explanation.) Any four of:
   - User awareness: the bank regularly informs customers that it never asks for credentials by SMS, so users recognise and ignore fake messages.
   - Two-factor authentication (2FA): even if a password is stolen, the attacker cannot log in without the one-time code sent to the customer's own phone.
   - Verification of the sender / official channels: customers check the bank's official number and type the website address themselves instead of clicking links, defeating the fake link.
   - Anti-spam / SMS filtering and takedown: the operator and bank report and block fraudulent numbers and fake websites, reducing the reach of the campaign.
   - Encryption (HTTPS) on the banking site: credentials typed on the genuine site travel encrypted, so they cannot be intercepted in transit.
3. (2 marks: 1 each.)
   Firewall: it filters traffic entering and leaving the bank's servers according to security rules, blocking unauthorised connection attempts and malicious traffic before it reaches the banking system.
   Encryption: protocols such as TLS (HTTPS) scramble the data exchanged between the customer and the bank, so even if an attacker intercepts the communication, the credentials and transaction details remain unreadable.
4. (1 mark.) The user is often the weakest link: most successful attacks (like phishing) exploit human error rather than technical flaws, so training users to recognise threats is an essential layer of security.

SOLUTION **Exercise 11 — SDLC essay and lab diagnosis (GCE style)**

Part A.

1. (4 marks: 1 per valid advantage/limitation.)

	Waterfall	Agile
Advantage 1	Simple, rigid structure: phases follow in order, easy to plan and manage.	Adapts easily to changing requirements through short iterations.
Advantage 2	Documentation is complete at each phase, useful for contracts and regulation.	Working software is delivered early and often; users give feedback continuously.
Limitation 1	Returning to a previous phase is very difficult and costly.	Requires constant customer involvement, which is not always available.
Limitation 2	The product is seen only at the end; errors in requirements are discovered late.	Less predictable schedule and documentation may be weaker.

2. (3 marks: 1 for the choice, 2 for three coherent arguments.)
Recommendation: the agile model. Arguments:
   - Changing user needs: the Cameroonian mobile-money market evolves fast and users' expectations are not fully known in advance; agile iterations allow the app to be adjusted after each user feedback.
   - Time to market: competition is strong; agile delivers a first working version quickly, then improves it, instead of waiting months for a complete product.
   - Regulation: financial regulations may change during development; agile lets the team integrate new compliance requirements in the next iteration rather than restarting the whole project.

Part B.

1. (4 marks: 1 per cause.) The four most probable causes:
   (a) Wrong IP configuration on one PC (wrong address, mask or duplicate address).
   (b) Faulty or disconnected cable between a PC and the switch (or switch port down).
   (c) A firewall on PC2 blocking ICMP (ping) requests.
   (d) The switch (or the relevant port) is faulty, misconfigured or powered off.
2. (1 mark.) Verification for each cause:
   (a) Run `ipconfig` on each PC to check the IP address and subnet mask.
   (b) Check the link LEDs on the switch ports and reseat/replace the cable; check port status on the switch.
   (c) Temporarily disable the firewall on PC2 (or allow ICMP) and ping again.
   (d) Test with another port or another switch; check that the switch is powered on.

WARNING **Examiner's expectations**

In Part A, marks are awarded for contrasted points (the same aspect discussed for both models) and for arguments explicitly linked to the given context, not generic definitions. In Part B, the diagnosis must be logical: since both PCs share one switch and one subnet, routing is not a possible cause — mentioning the router or the gateway loses credibility.

14 SUMMARY SHEET

Networks, Security & Software Engineering

Data Communication
- Signals: analog / digital
- Media: cable, fibre, radio
- Modes: simplex, duplex

Network Fundamentals
- LAN, MAN, WAN
- Topologies: bus, star, ring
- OSI & TCP/IP models

IP Addressing
- Classes A–E, IPv4
- Subnetting, masks, CIDR

Security
- CIA triad
- Threats, malware, firewall
- Encryption, authentication

Software Engineering
- SDLC phases
- Models: waterfall, agile
- Testing & maintenance

Theoretical CS
- Automata, Turing machine
- Computability, complexity

Mind map of the chapter: networking, security, software engineering and theoretical computer science.

FICHE BILAN **L'essentiel à connaître pour l'examen**

Key definitions.

- **Data communication**: exchange of data between two devices via a transmission medium, governed by protocols. Effectiveness rests on delivery, accuracy, timeliness and jitter.
- **Signal**: electrical or electromagnetic representation of data; analog (continuous) or digital (discrete).
- **Bandwidth**: range of frequencies a medium can carry (Hz); **bit rate**: number of bits transmitted per second (bit/s).
- **Network**: set of interconnected devices sharing resources. LAN (building), MAN (city, e.g. a university campus network in Yaoundé), WAN (country/world, e.g. the Internet).
- **Topology**: physical/logical layout: bus, star, ring, mesh. Star is the most common in LANs (switch at the centre).
- **Protocol**: rules governing communication (TCP, IP, HTTP, FTP, SMTP, DNS, DHCP).

- **IP address**: logical identifier of a host; IPv4 = 32 bits written as 4 dotted-decimal octets (e.g. 192.168.1.10).
- **Subnet mask**: separates the network part from the host part (e.g. 255.255.255.0 = /24).
- **Firewall**: hardware/software filtering traffic according to security rules.
- **Encryption**: transforming plaintext into ciphertext with a key; symmetric (one key) vs asymmetric (public/private keys).
- **Software engineering**: systematic, disciplined approach to developing, operating and maintaining software.
- **SDLC**: Software Development Life Cycle: analysis → design → implementation → testing → deployment → maintenance.

Laws, formulas and results to know.

- Shannon–Hartley (capacity of a noisy channel): $C = B \log_2(1 + S/N)$ bits/s, where B is the bandwidth and S/N the signal-to-noise ratio.
- Nyquist (noiseless channel): $C = 2B \log_2 M$ bits/s, with M signal levels.
- Number of addresses in a /n network: 2^{32-n}; usable hosts: $2^{32-n} - 2$ (network and broadcast addresses excluded).
- Network address = IP AND mask (bitwise); broadcast = network address with host bits set to 1.
- IPv4 classes: A (1–126, /8), B (128–191, /16), C (192–223, /24); D = multicast, E = experimental. Private ranges: 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16.
- OSI model (7 layers, bottom-up): Physical, Data Link, Network, Transport, Session, Presentation, Application. TCP/IP model: 4 layers (Link, Internet, Transport, Application).
- Complexity classes: P (solvable in polynomial time), NP (verifiable in polynomial time); the halting problem is undecidable (Turing).

Methods.

1. Subnetting: (1) write the mask in binary; (2) count borrowed host bits $b \Rightarrow 2^b$ subnets; (3) block size = 256 − mask value in the interesting octet; (4) list subnet addresses in steps of the block size; (5) usable range = between network and broadcast addresses.
2. Security analysis: identify assets → threats (virus, worm, Trojan, phishing, DoS) → apply CIA triad (Confidentiality, Integrity, Availability) → countermeasures (firewall, antivirus, strong passwords, encryption, backups, updates).
3. Choosing an SDLC model: waterfall for fixed, well-understood requirements; iterative/agile (e.g. Scrum) when requirements evolve; V-model when testing is critical.

4. Testing: unit → integration → system → acceptance (UAT). Distinguish verification (built right?) from validation (right product?).

Common mistakes.

- Confusing bandwidth (Hz) with bit rate (bit/s); confusing MB and Mb.
- Counting the network and broadcast addresses as usable hosts.
- Mixing up OSI layer order, or placing IP at the Transport layer (IP = layer 3, TCP/UDP = layer 4).
- Believing a firewall alone guarantees security; forgetting the human factor (phishing).
- Writing 255.255.255.255 as a valid host address; using non-contiguous masks.
- Skipping the analysis phase and coding immediately; confusing white-box and black-box testing.
- Claiming every problem is computable: the halting problem is not.

GCE tips.

- In subnetting questions, always show the binary working of the mask and the AND operation — method marks are awarded.
- Learn one labelled diagram for each: OSI model, star topology, SDLC cycle, finite automaton.
- Define every term before using it ("A firewall is…"), then give one concrete example (e.g. securing a school network in Bamenda).
- For essays on security or software models, structure: definition → explanation → example → advantage/limitation.
- Manage time: a trace table or subnet calculation must be checked twice; a wrong AND ruins the whole answer.

Signed • OnBuch+
ONBUCH+ TEACHING TEAM
FOLLOWS THE OFFICIAL MINESEC SYLLABUS

OnBuch+